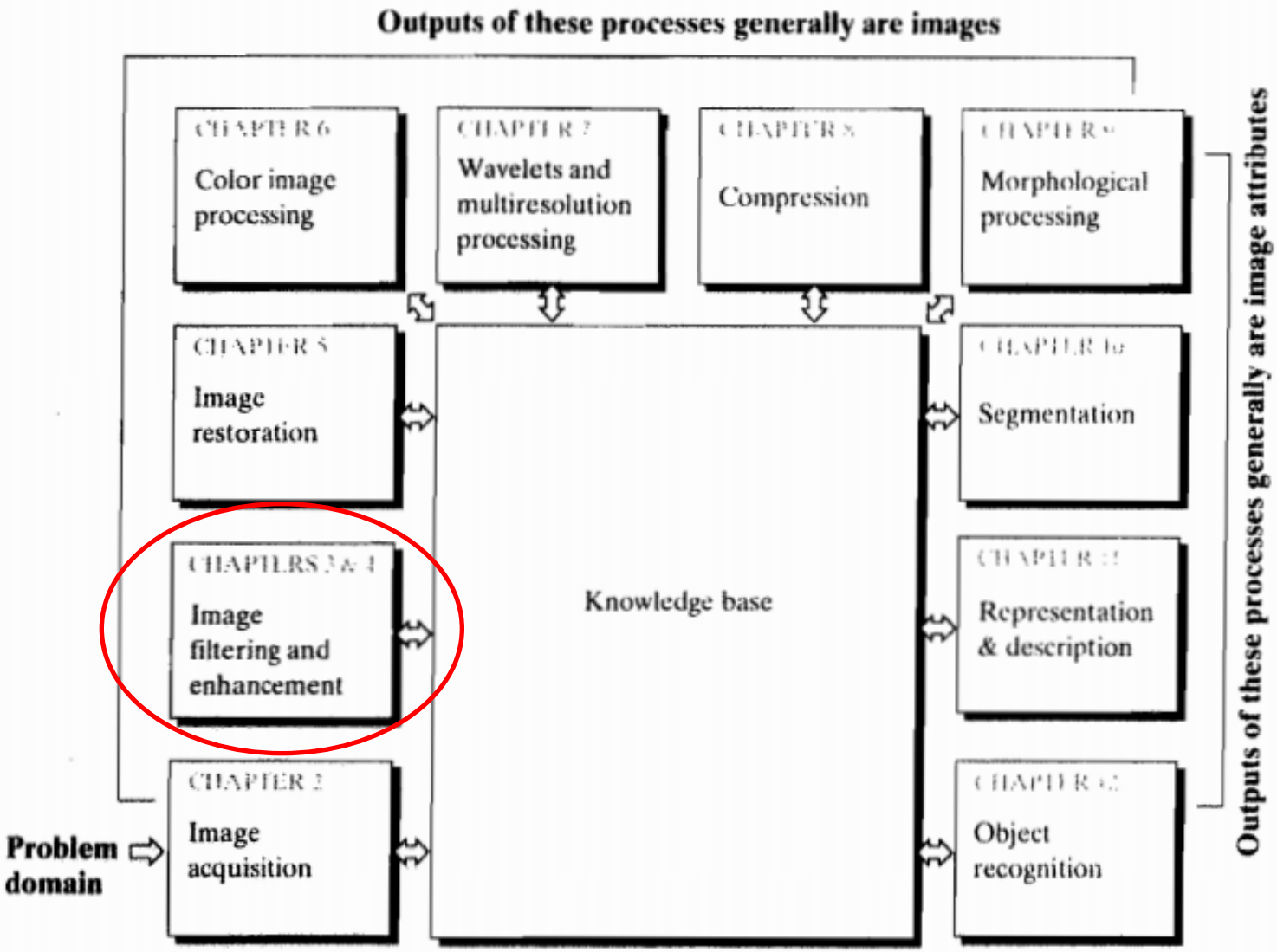
Chapter 3
Intensity Transformations and Spatial Filtering

Remember?

FIGURE 1.23 Fundamental steps in digital image processing. The chapter(s) indicated in the boxes is where the material described in the box is discussed.



Our Objective

Image Enhancement

- Enhancement is the process of manipulating an image so that the result is more suitable than the original for a specific application.
- Very subjective
- Application dependent
- Viewer is the ultimate judge

Classification of Image Enhancement

- Enhancement in spatial domain (Chapter 3)

Spatial domain is a plane where a digital image is defined by the spatial coordinates of its pixels.

- Enhancement in frequency domain (Chapter 4)

Frequency domain where a digital image is defined by its decomposition into spatial frequencies participating in its formation.

Classification of Image Enhancement

- **Enhancement in spatial domain**
  - The section of the real plane spanned by the coordinates of an image is called the spatial domain.
  - manipulates pixel by pixel basis

How does It Work?

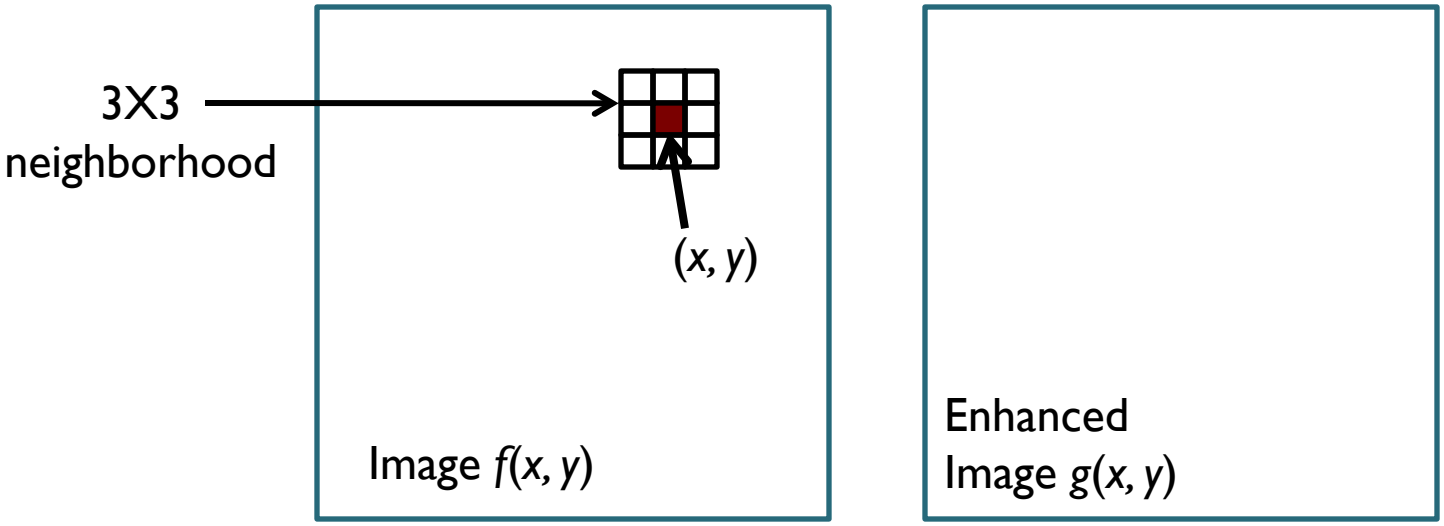
Enhancement in Spatial Domain

- The spatial domain processes can be denoted by the expression

$$g(x, y) = T\,[\,f(x, y)]$$

- f(x, y) is the input image and g(x, y) is the output image and T is an operator defined over a neighborhood of point (x, y)

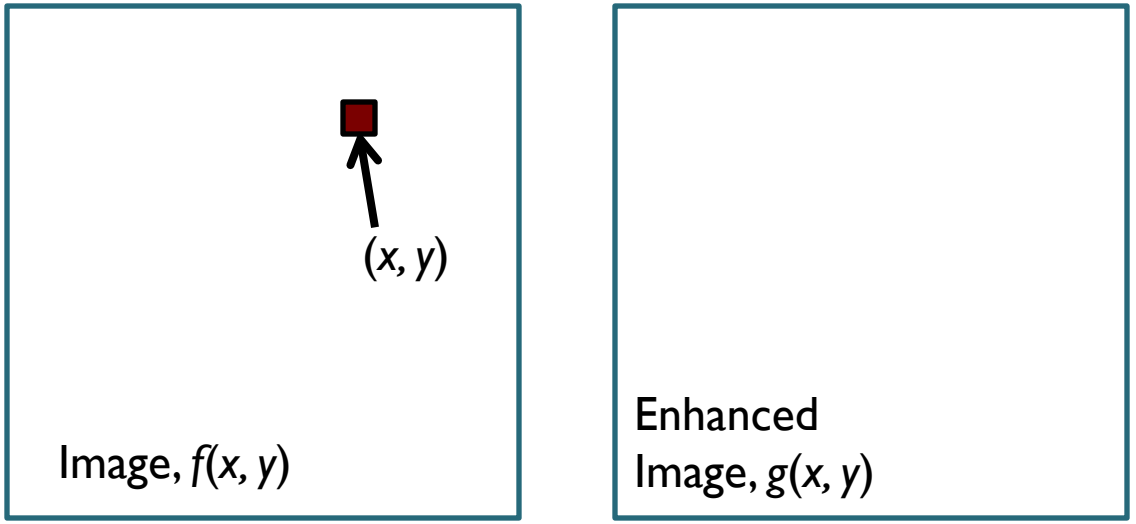
Enhancement in Spatial Domain



$$g(x,y) = T\left[f(x,y)\right]$$

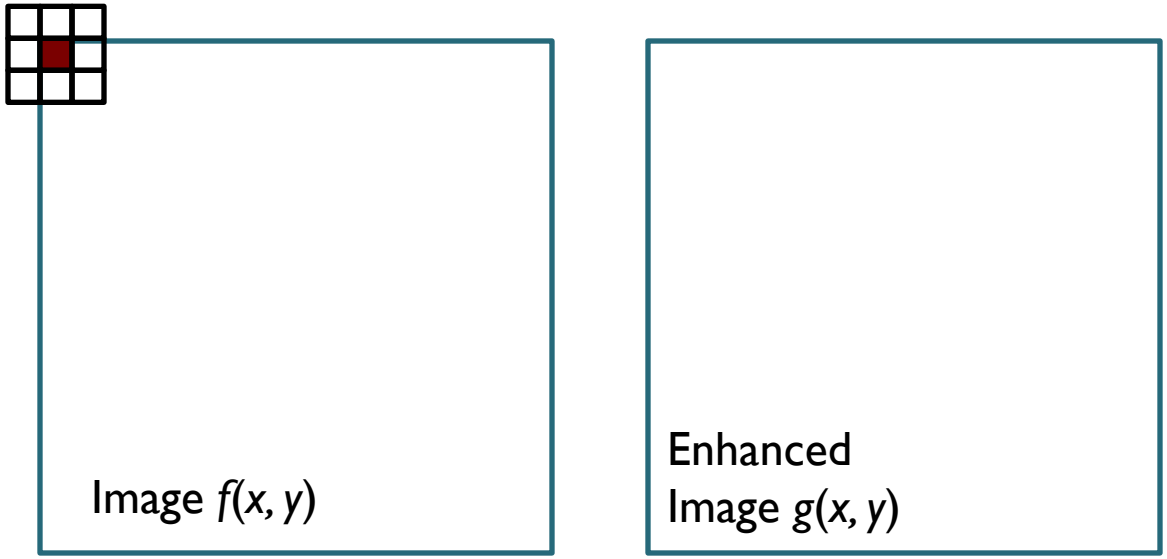
Enhancement in Spatial Domain

- Neighborhood can be as small as a single pixel



Enhancement in Spatial Domain

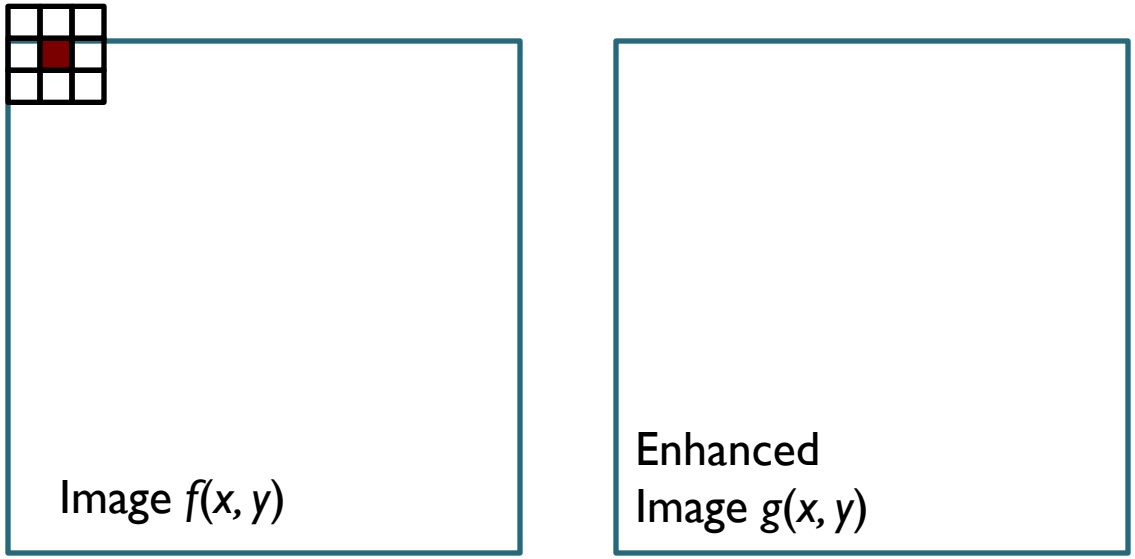
- Neighborhood (sub-image) rolls over the entire image, each time centering a different pixel.
- For any specific location (x, y), the value of the output image g at those coordinates is equal to the result of applying T to the neighborhood with origin at (x, y).



$$g(x,y) = T[f(x,y)]$$

Enhancement in Spatial Domain

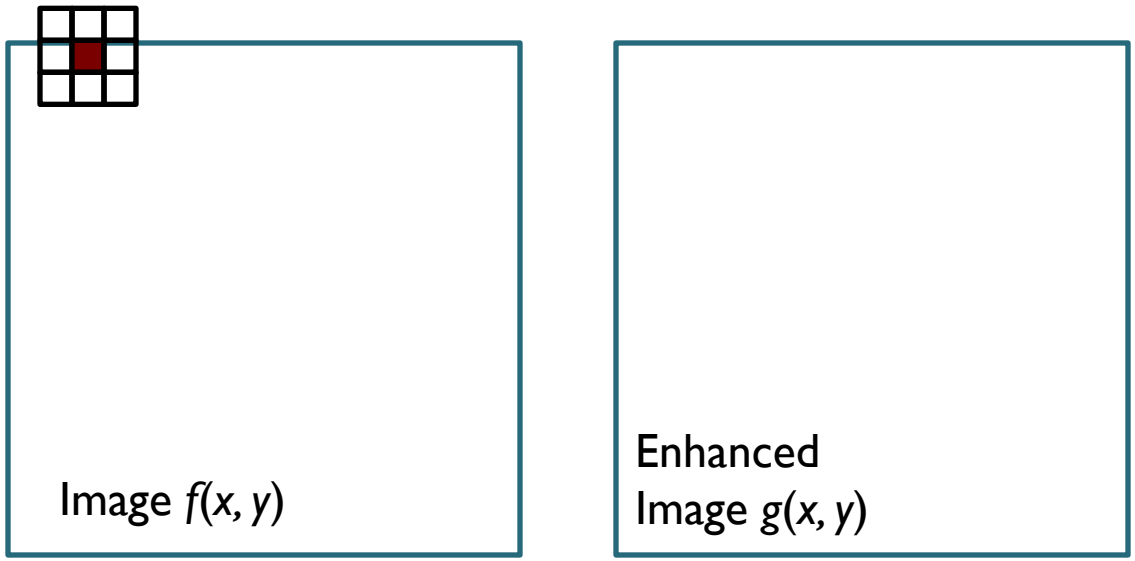
- Neighborhood (sub-image) rolls over the entire image, each time centering a different pixel.
- For any specific location (x, y), the value of the output image g at those coordinates is equal to the result of applying T to the neighborhood with origin at (x, y).



$$g(x,y) = T[f(x,y)]$$

Enhancement in Spatial Domain

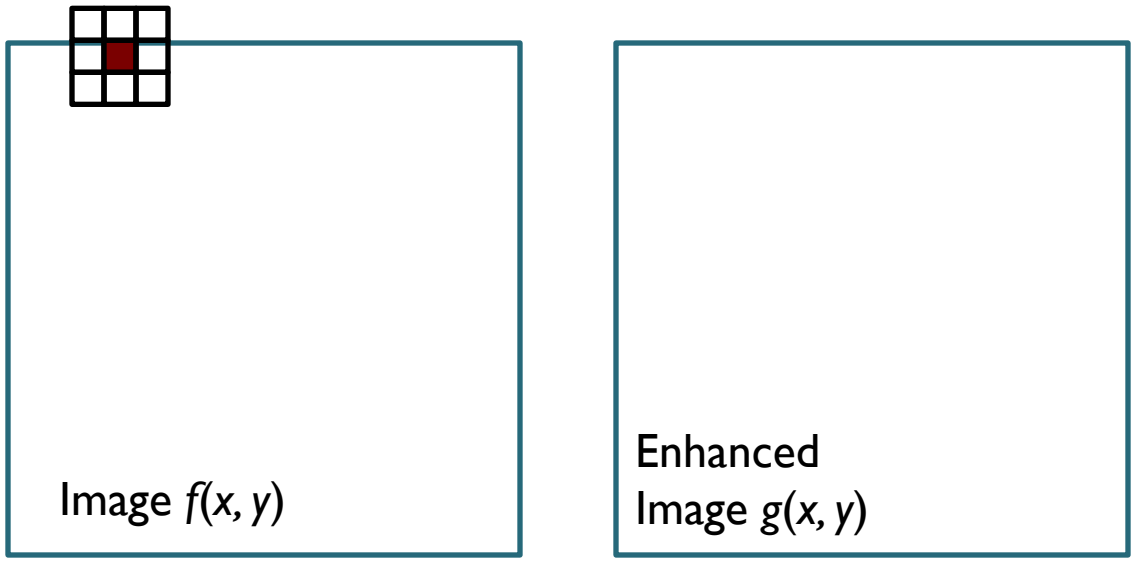
- Neighborhood (sub-image) rolls over the entire image, each time centering a different pixel.
- For any specific location (x, y), the value of the output image g at those coordinates is equal to the result of applying T to the neighborhood with origin at (x, y).



$$g(x,y) = T[f(x,y)]$$

Enhancement in Spatial Domain

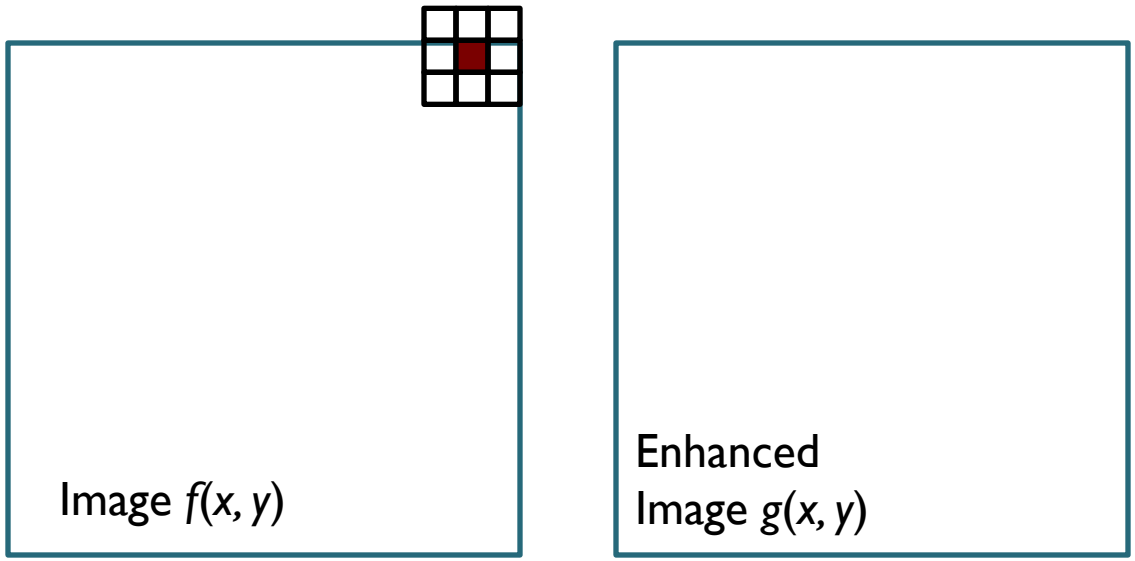
- Neighborhood (sub-image) rolls over the entire image, each time centering a different pixel.
- For any specific location (x, y), the value of the output image g at those coordinates is equal to the result of applying T to the neighborhood with origin at (x, y).



$$g(x,y) = T[f(x,y)]$$

Enhancement in Spatial Domain

- Neighborhood (sub-image) rolls over the entire image, each time centering a different pixel.
- For any specific location (x, y), the value of the output image g at those coordinates is equal to the result of applying T to the neighborhood with origin at (x, y).



$$g(x,y) = T[f(x,y)]$$

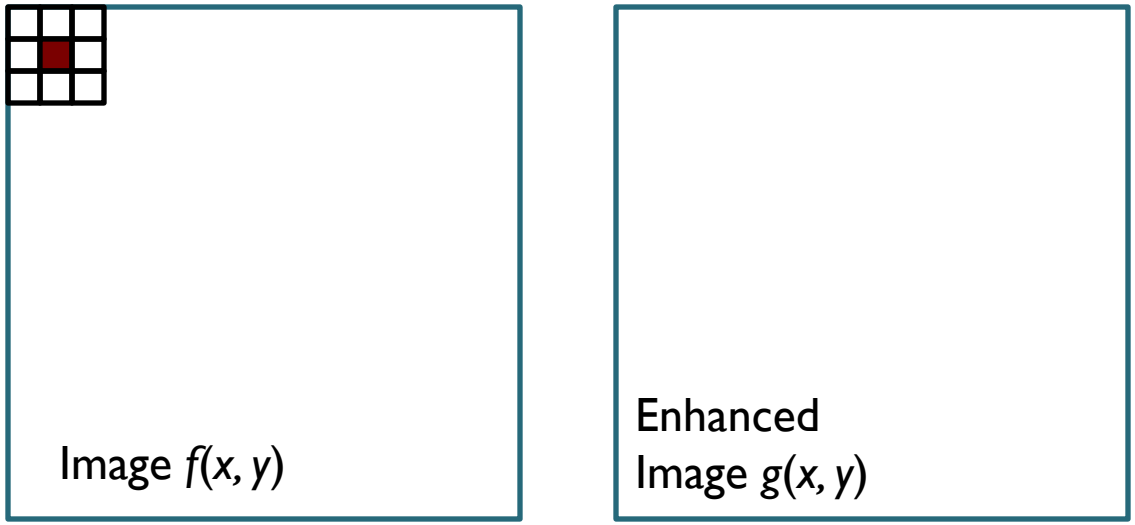
Enhancement in Spatial Domain

- Neighborhood (sub-image) rolls over the entire image, each time centering a different pixel.
- For any specific location (x, y), the value of the output image g at those coordinates is equal to the result of applying T to the neighborhood with origin at (x, y).

$$g(x,y) = T[f(x,y)]$$

Enhancement in Spatial Domain

- Neighborhood (sub-image) rolls over the entire image, each time centering a different pixel.
- For any specific location (x, y), the value of the output image g at those coordinates is equal to the result of applying T to the neighborhood with origin at (x, y).



$$g(x,y) = T[f(x,y)]$$

Spatial Filtering

- This procedure is called Spatial Filtering.
- The neighborhood along with a predefined operation is called a spatial filter.
- Spatial filter is also known as spatial mask, kernel, template or window. Spatial mask is the most popular.

Intensity Transformation Function

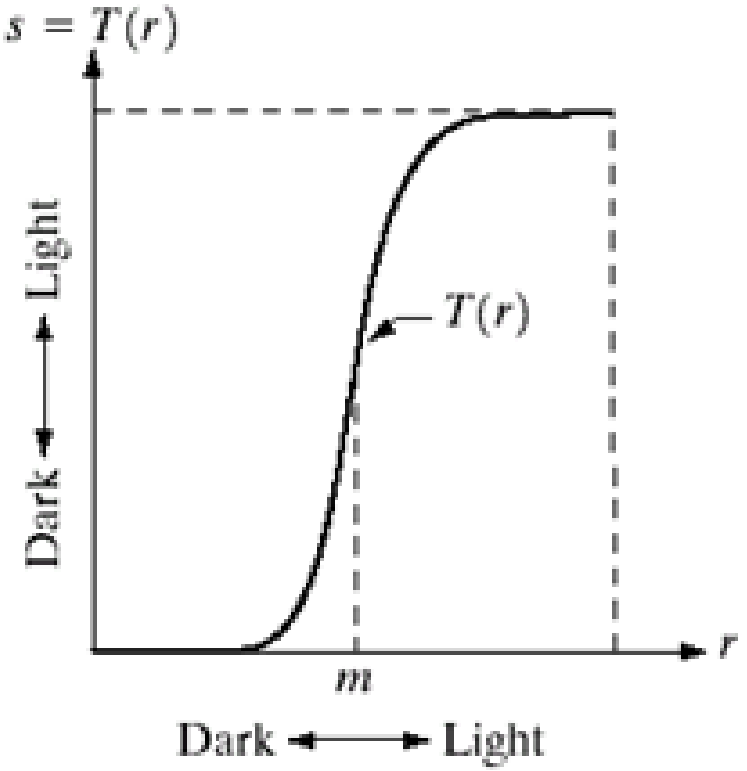
Intensity Transformation Function

- If the neighborhood is of size 1×1, g depends only on the value of f at a single point (x, y) and T becomes intensity (gray-level) transformation function of the form

$$s = T(r)$$

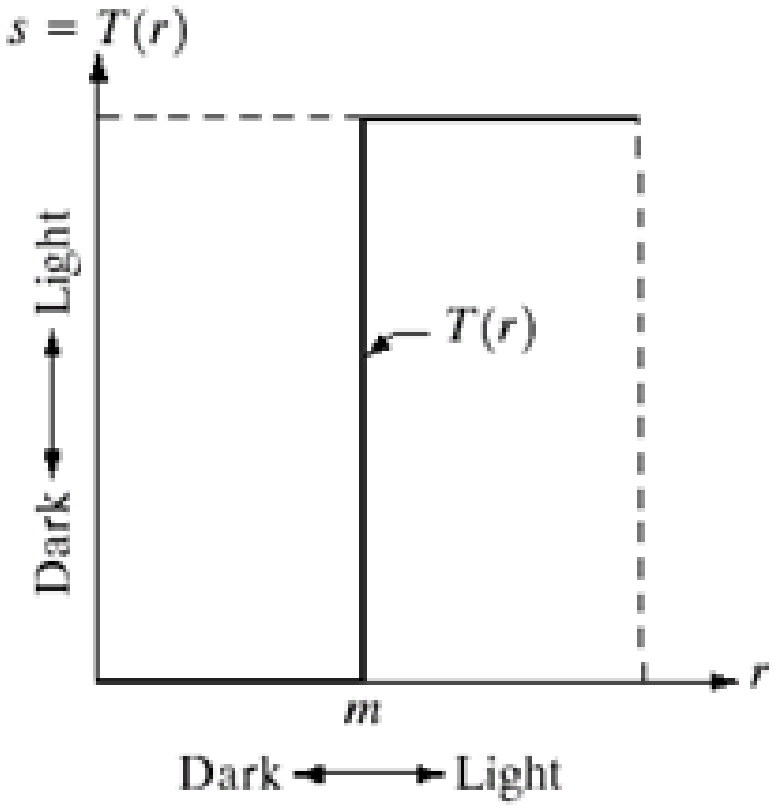
Contrast Stretching

- Input gray level R
- Produce an image of higher contrast than the original.
- Below a threshold (*m*), image is compressed towards black
- Values of r lower than m are compressed by the transformation function into a narrow range of s, toward black.
- Above *m*, image is stretched towards white

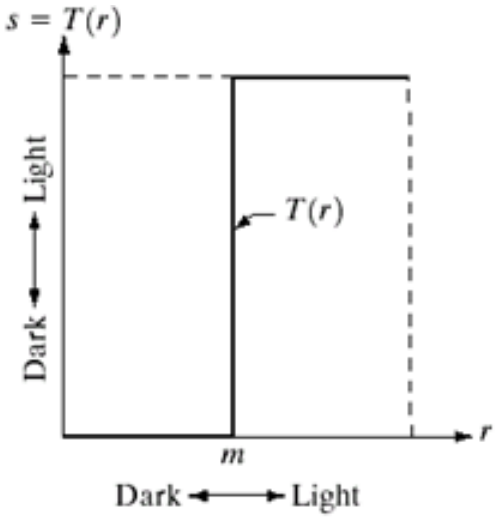


Thresholding

- Thresholding is a special case of contrast stretching.
- Produces a binary image
- Useful in image segmentation



Thresholding



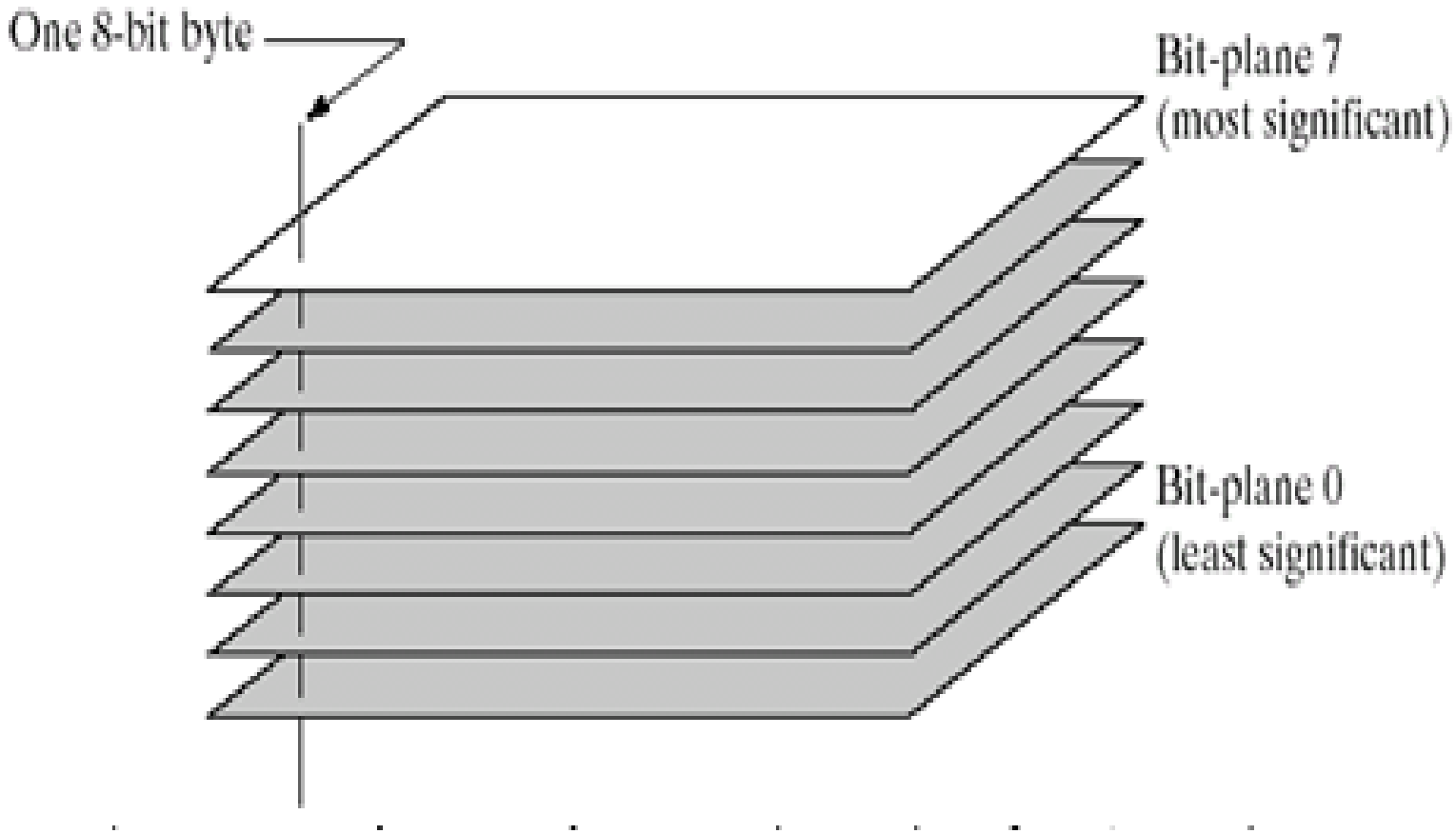
r

s

Bit Plane Slicing

What is this all about ???

Bit-plane Slicing



Bit-plane Slicing

Bit-plane Slicing

- The four higher order bit, especially the last two, contain a significant amount of visually significant data.
- The lower-order planes contribute to more subtle intensity details in the image.

original image

Bit-plane Slicing

Some of the bit plane sliced images have black border.

Look at this gray border. Its decimal value is 194

Some of the bit plane sliced images have white border.

original image

Bit-plane Slicing

Binary of 194 is 11000010

original image

Bit-plane Slicing

Binary of 194 is 1100001**0**

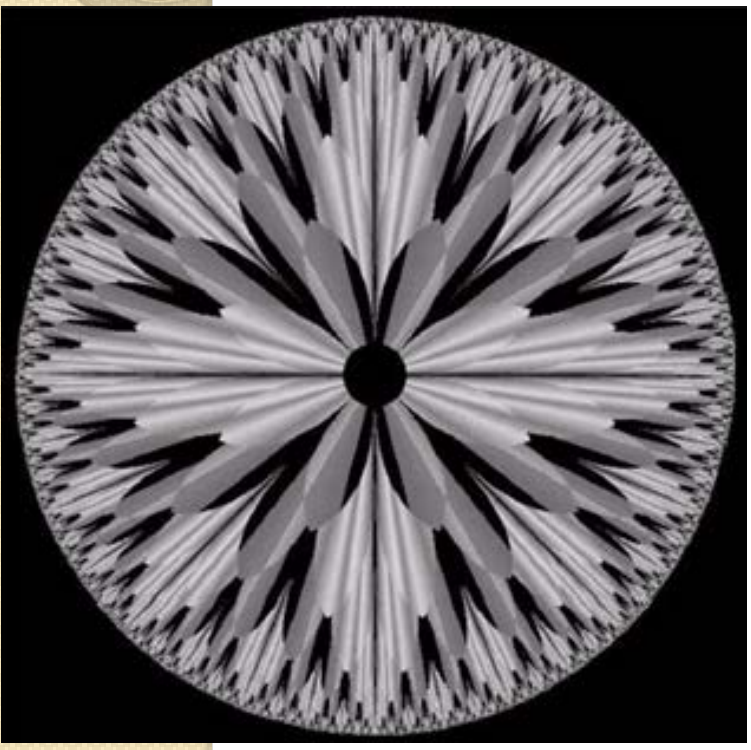
original image

Bit-plane Slicing

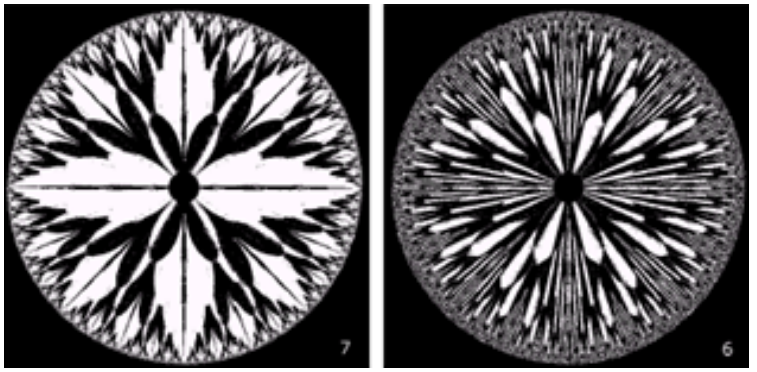
Binary of 194 is 110000**1**0

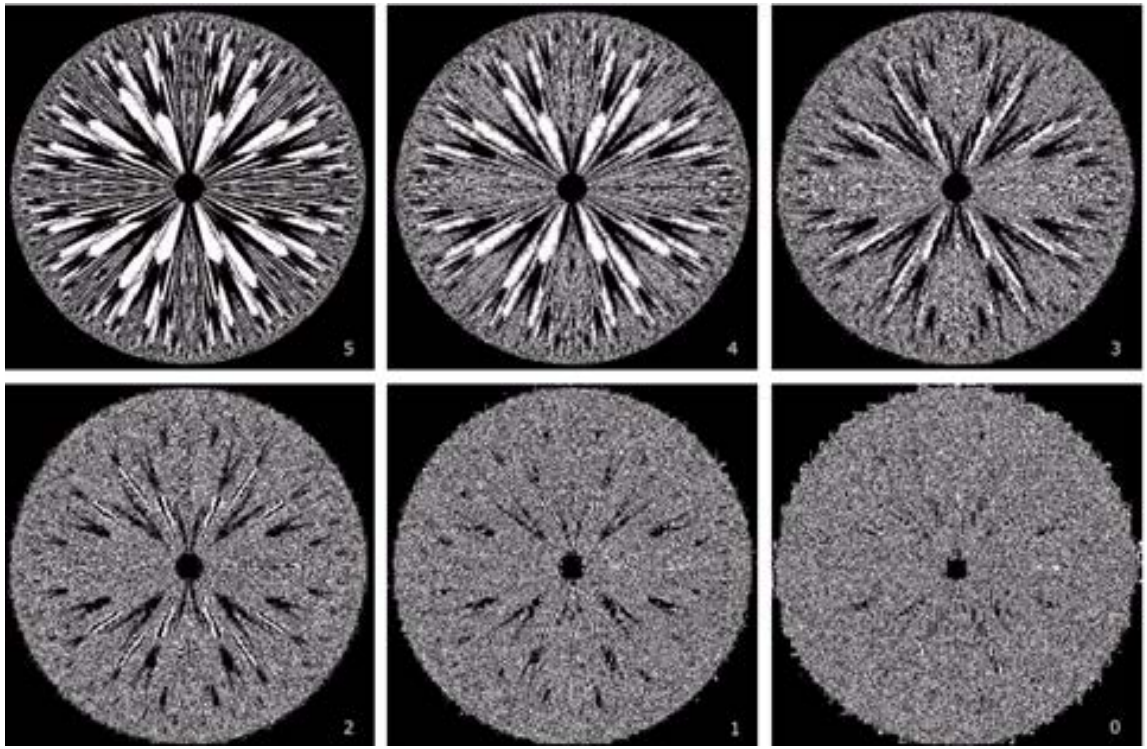
original image

Bit-plane Slicing



Original Image





Bit-plane Slicing

- Useful for analyzing the relative importance of each bit in an image.
- This process aids in determining the adequacy of the number of bits used to quantize the image.
- Useful in image compression.

Bit-plane Slicing

Image Compression

Bit planes 8 & 7

Bit planes 8, 7, 6 & 5

Bit planes 8, 7 & 6

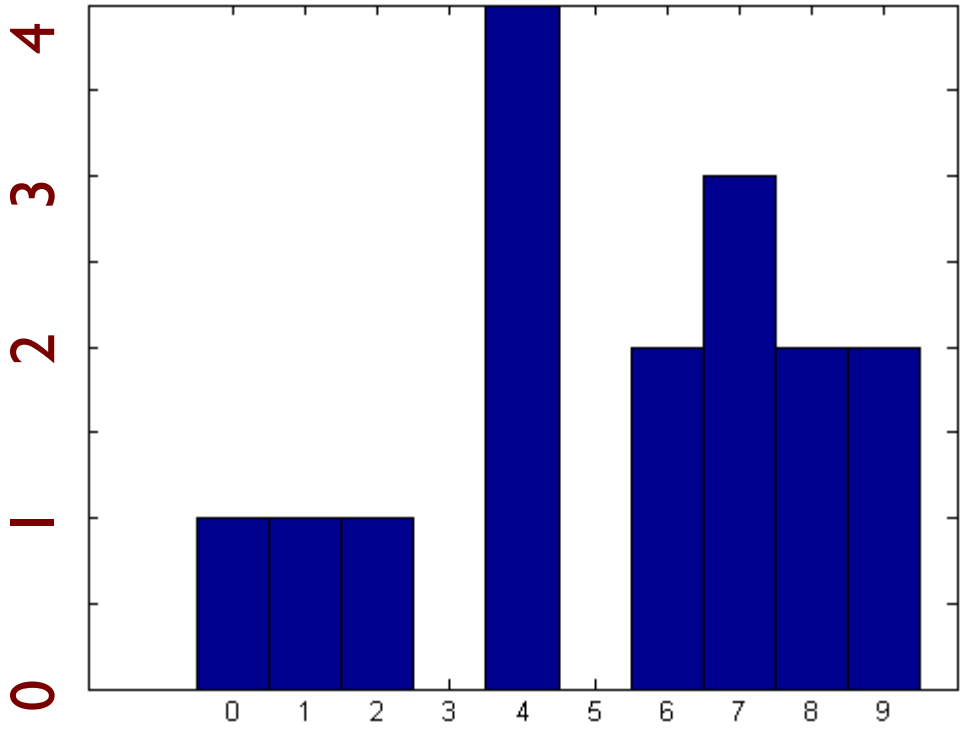
Original Image

Image Enhancement using Histogram Processing

Image Histogram

9	8	9	8
2	7	4	7
6	4	6	1
4	0	7	4

An image



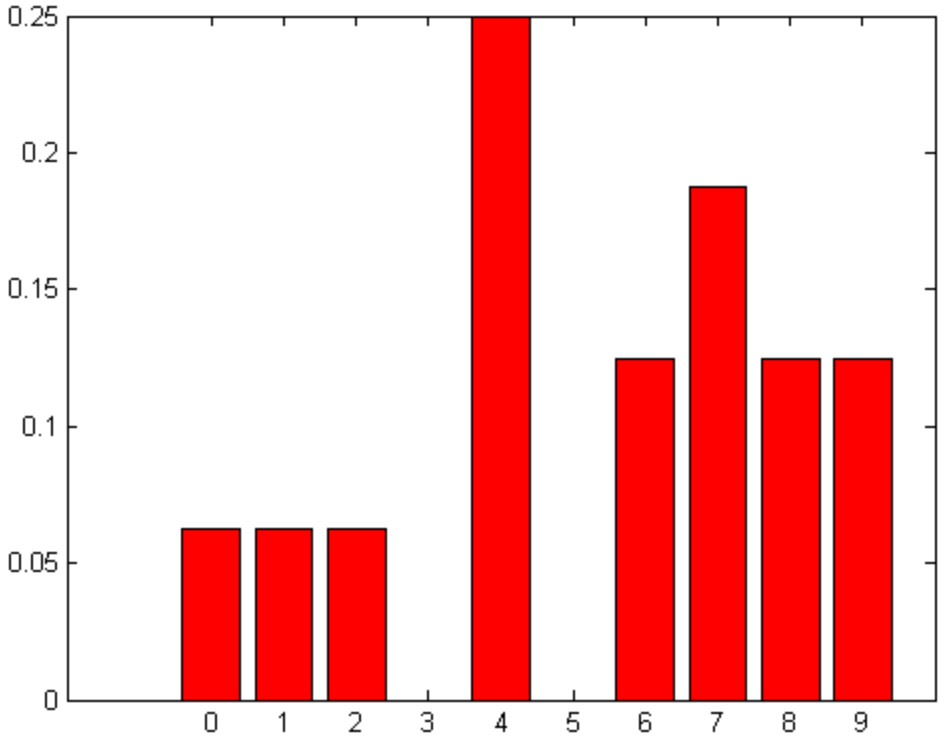
Histogram:

	0	1	2	3	4	5	6	7	8	9
h:	1	1	1	0	4	0	2	3	2	2

Normalized Image Histogram

9	8	9	8
2	7	4	7
6	4	6	1
4	0	7	4

An image



Histogram:

	0	1	2	3	4	5	6	7	8	9
h:	?	?	?	0	?	0	?	?	?	?

Normalized Image Histogram

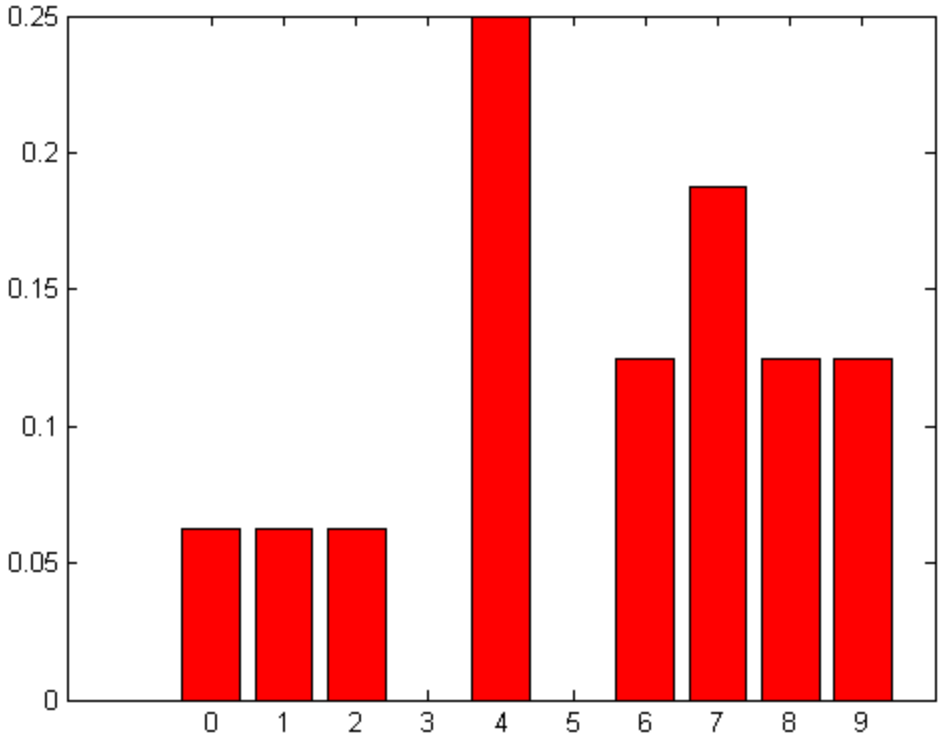
$$p(r_k) = n_k / n$$

- Normalized histogram is more like probabilities
- $p(r_k)$: how likely a pixel will have gray level r_k

Normalized Image Histogram

9	8	9	8
2	7	4	7
6	4	6	1
4	0	7	4

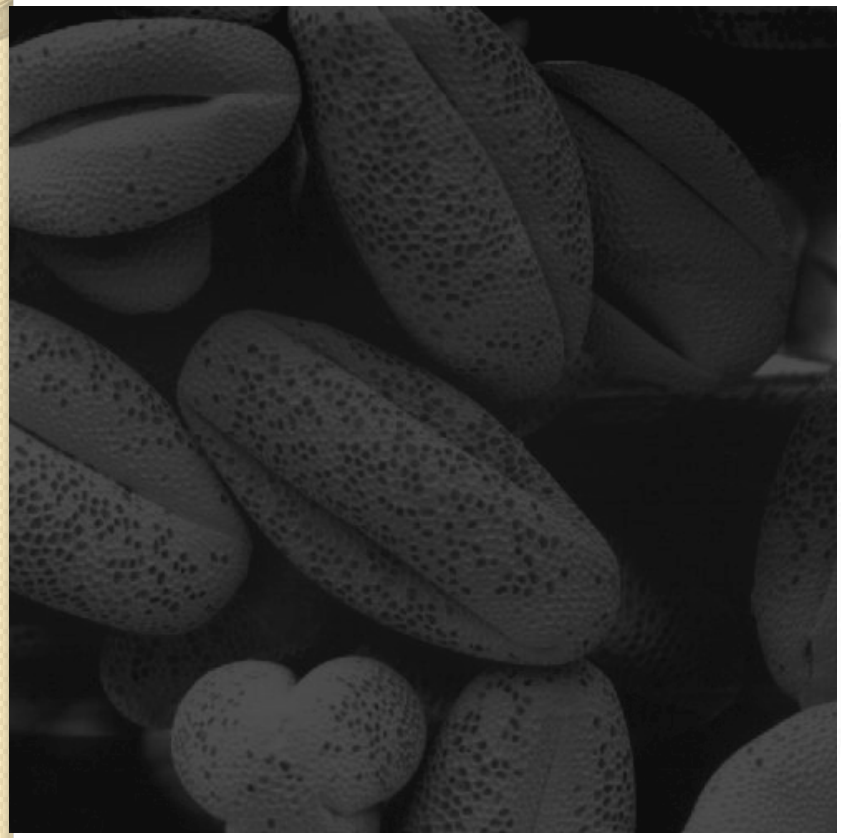
An image

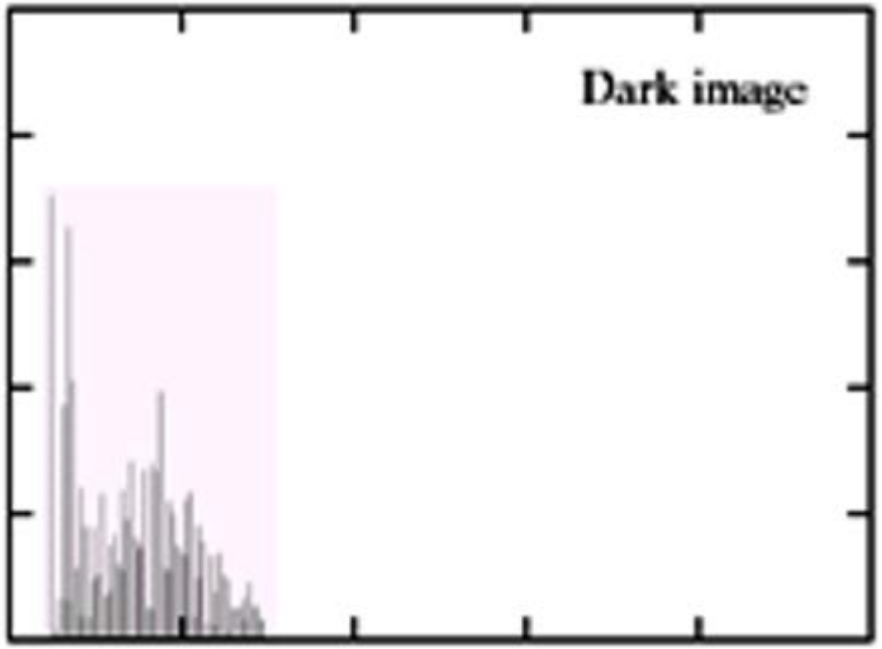


Histogram:

	0	1	2	3	4	5	6	7	8	9
h:	1/16	1/16	1/16	0	1/4	0	1/8	3/16	1/8	1/8

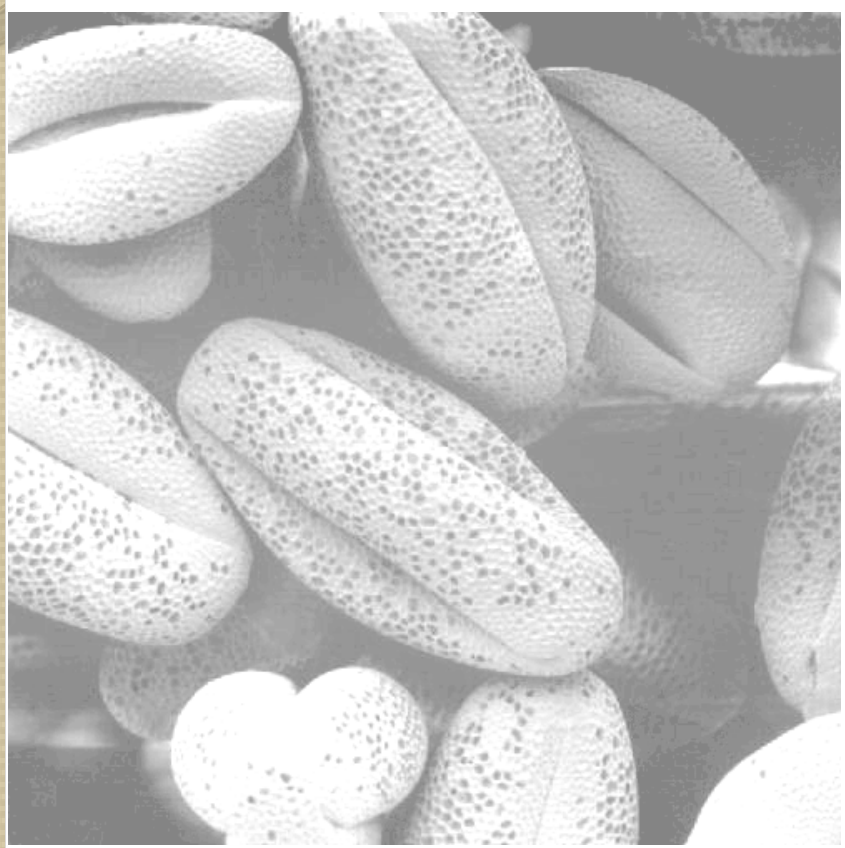
Image Histogram

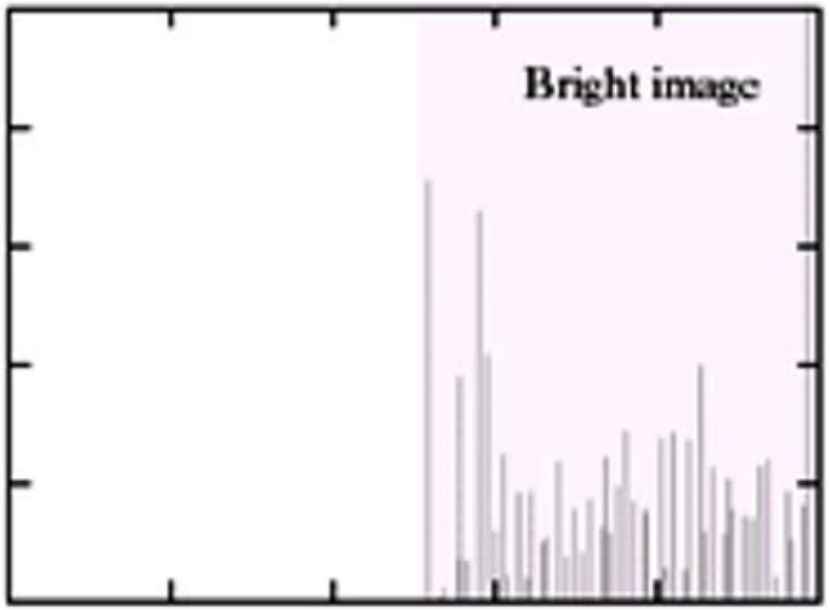




Histogram for dark image

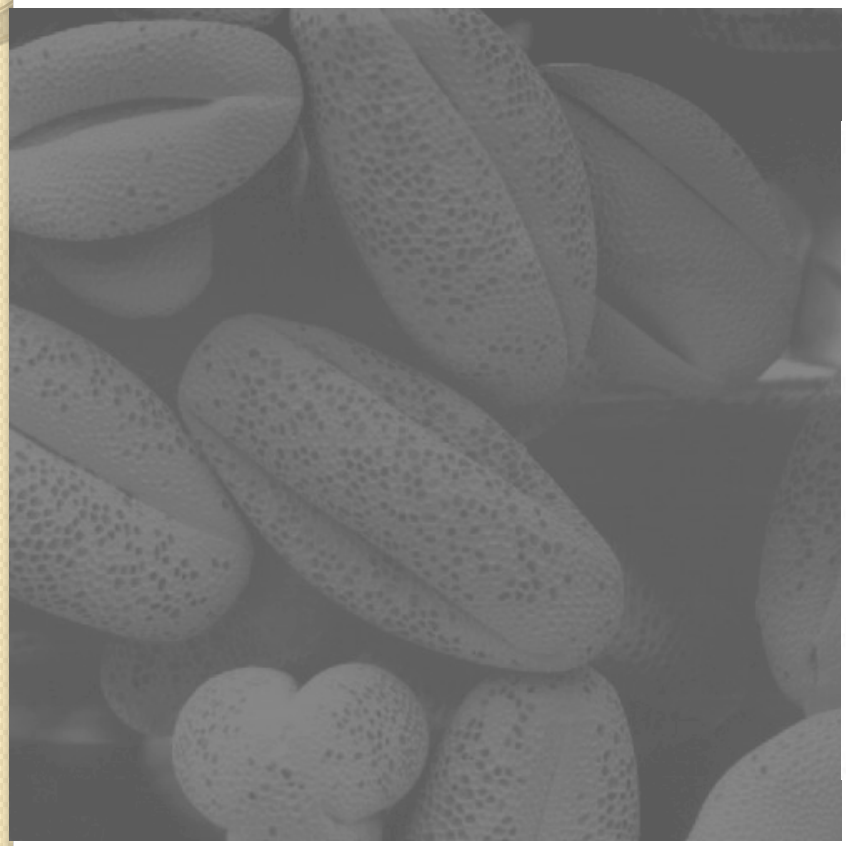
Image Histogram

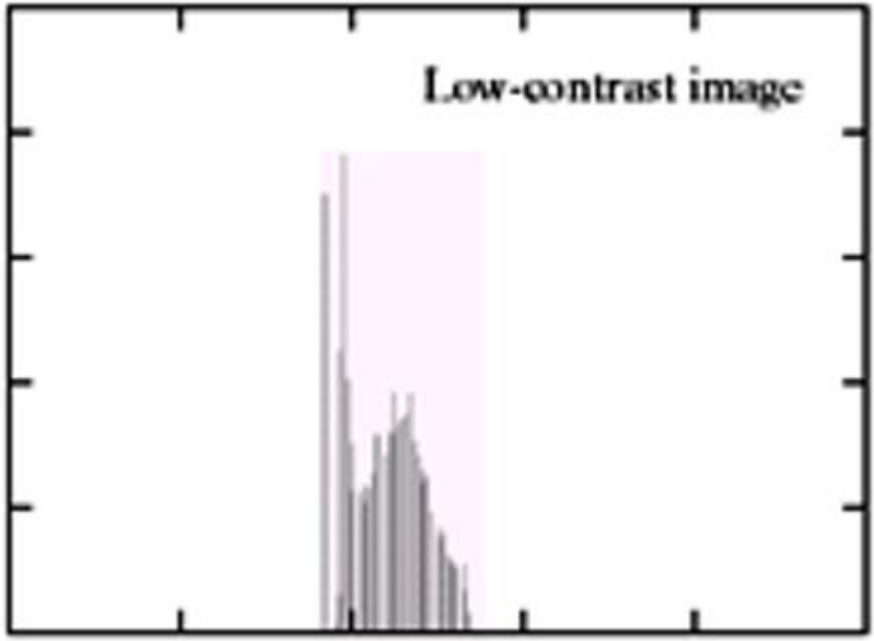




Histogram for bright image

Image Histogram





Histogram for low contrast image

Image Histogram

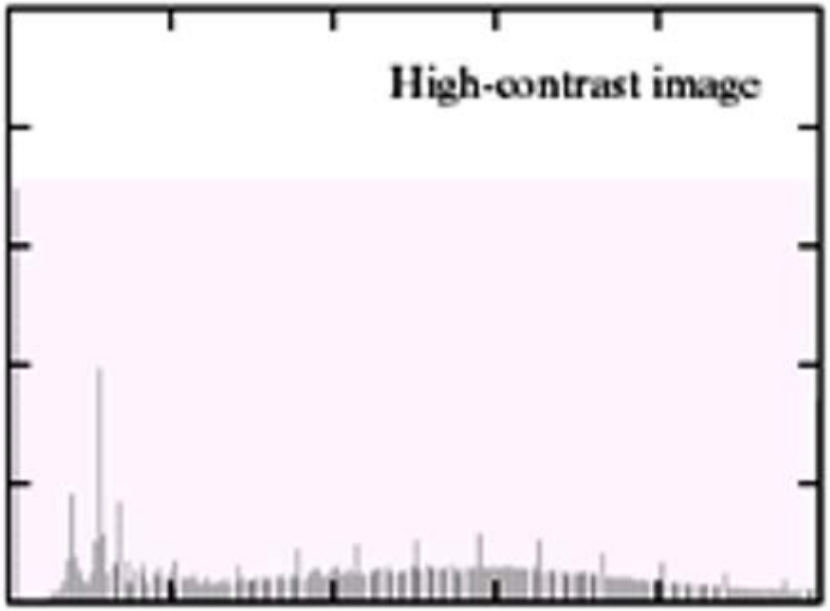
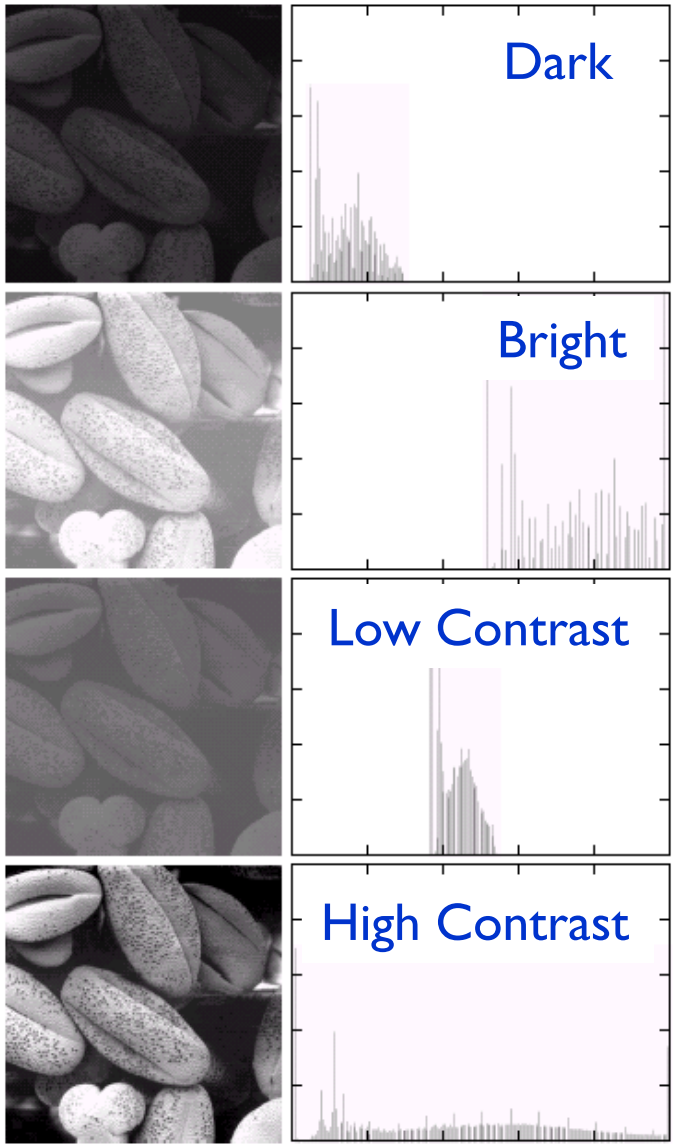


Image Histogram



- **Dark images:** bins in the low end
- **Bright images:** bins in the high end
- **Low contrast images:** bins in a narrow range
- **High contrast images:** bins are in the *entire range*

Image Histogram

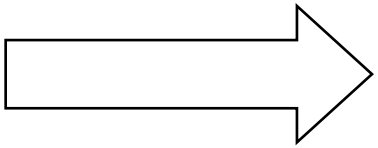
- An image whose pixels tend to occupy the entire range of possible intensity level, will have an appearance of high contrast.

Image Histogram

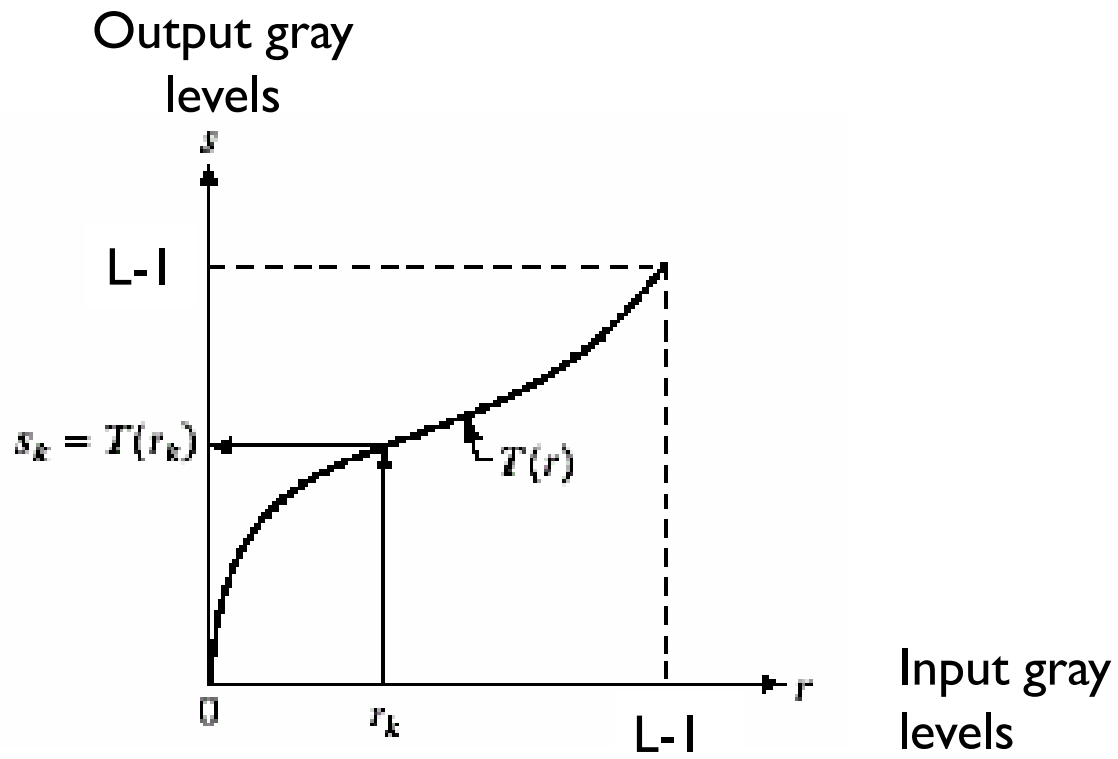
- histogram(S,'Normalization','probability')

Histogram Equalization

Histogram Equalization



Histogram Equalization

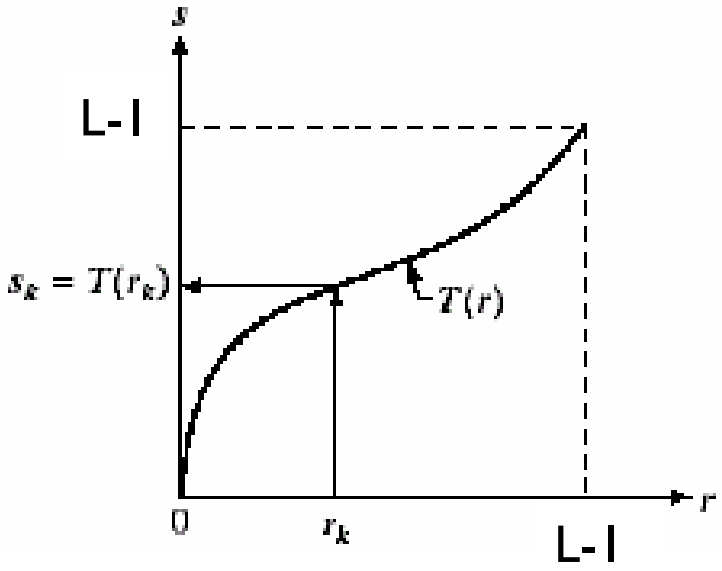


Histogram Equalization

- Let *s*, *r*: are all discrete and in [0, *L*-1]

$$s = T(r) \qquad 0 \le r \le \mathrm{L}\text{-}1$$

- Let, 2 conditions hold:
- *T(r): single valued and monotonous in* $0 \le r \le \mathrm{L}\text{-}1$
- $0 \le T(r) \le L-1 \quad \text{for } 0 \le r \le L-1$



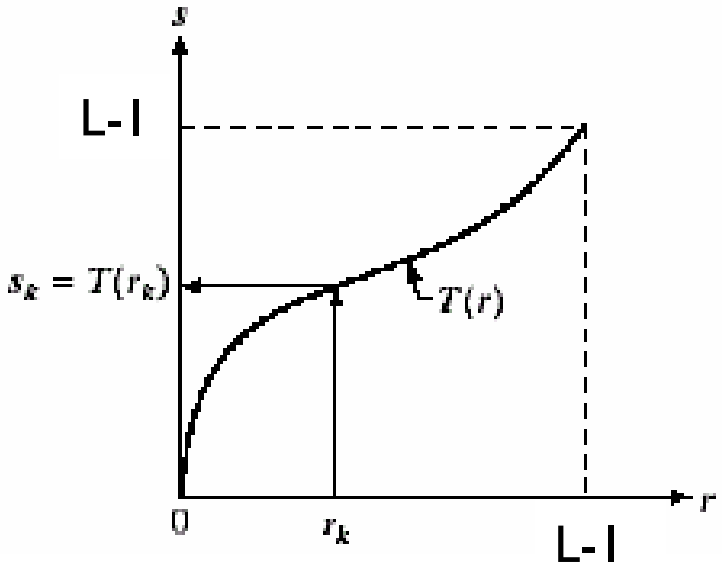
Condition 1 guarantees that output intensity values will never be less than corresponding input values thus preventing artifacts created by reversals of intensity.

Histogram Equalization

- Let *s*, *r*: are all discrete and in [0, *L*-1]

$$s = T(r) \qquad 0 \le r \le \mathrm{L}\text{-}1$$

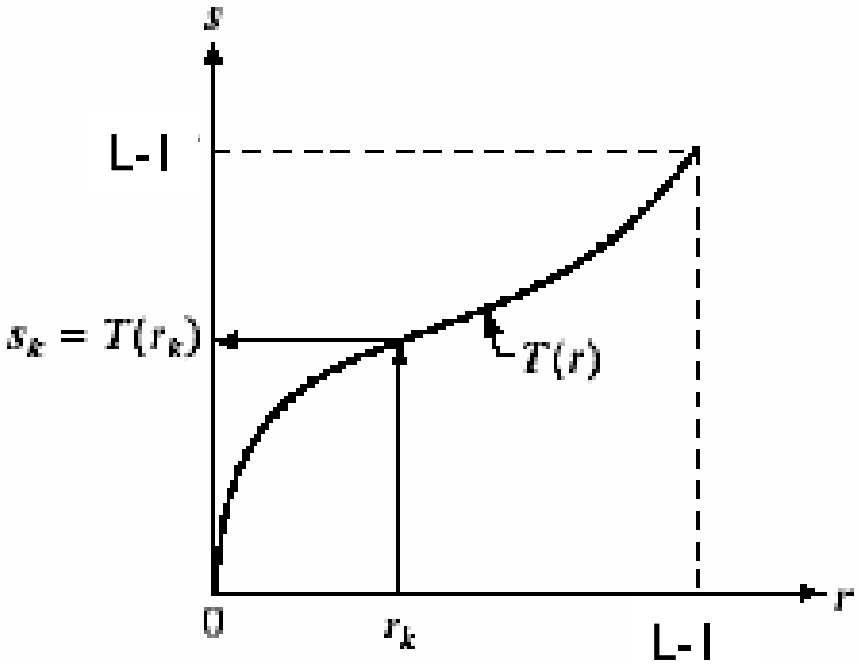
- Let, 2 conditions hold:
- *T(r): single valued and monotonous in* $0 \le r \le \mathrm{L}\text{-}1$
- $0 \le T(r) \le L-1 \quad \text{for } 0 \le r \le L-1$



Condition 2 guarantees that the range of output intensities is the same as the input.

Histogram Equalization

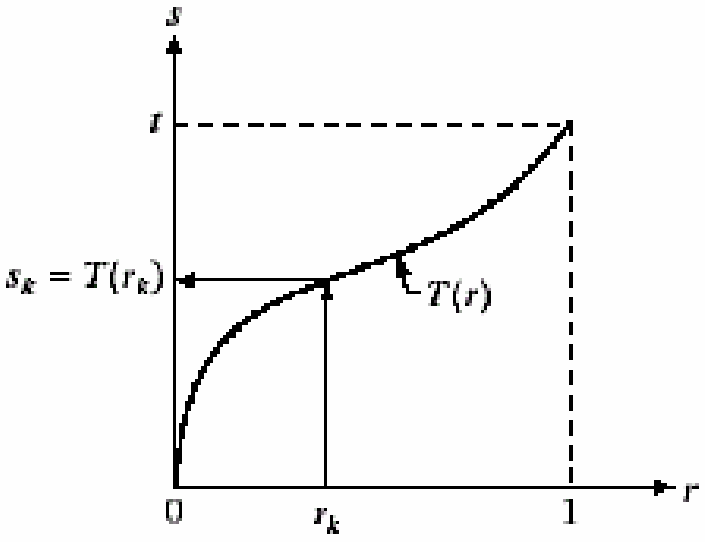
- Let,
  - $p_r(r)$: the equalized histogram of the given image
  - $p_s(s)$: the equalized histogram of the output image



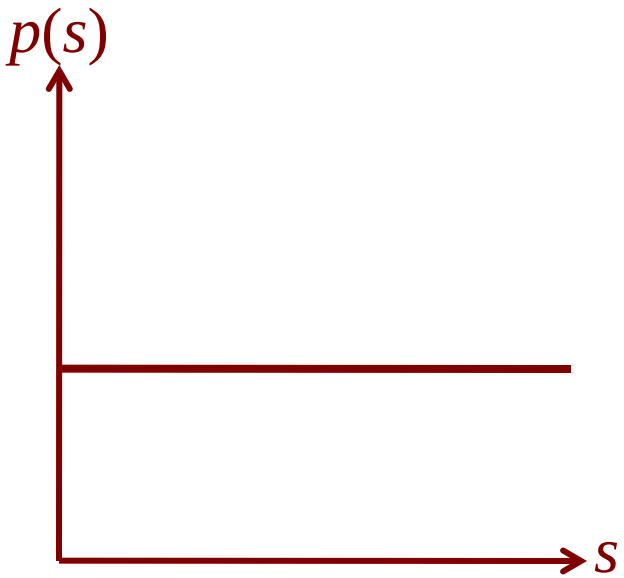
Histogram Equalization

- **Objective:** to find the transformed image so that $p_s(s)$ of the transformed image is uniform

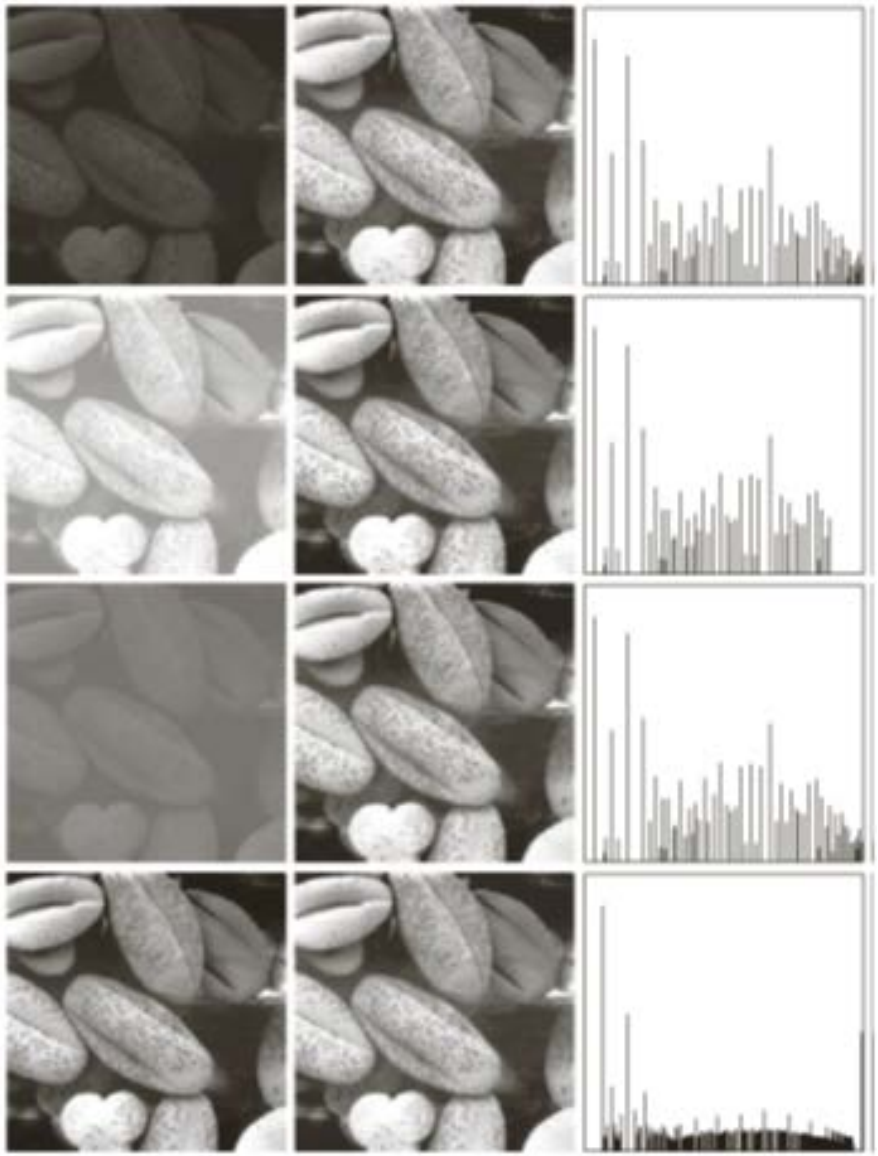
Apply this



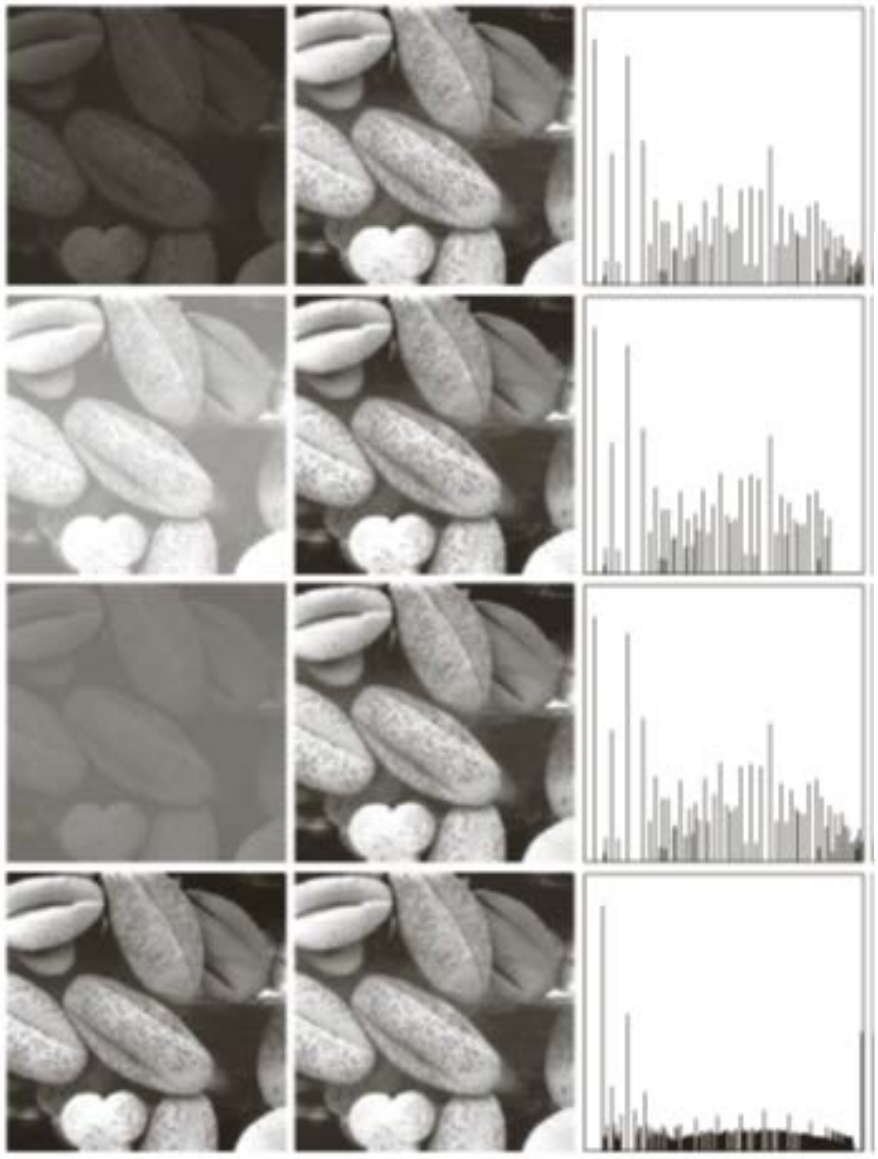
Get this

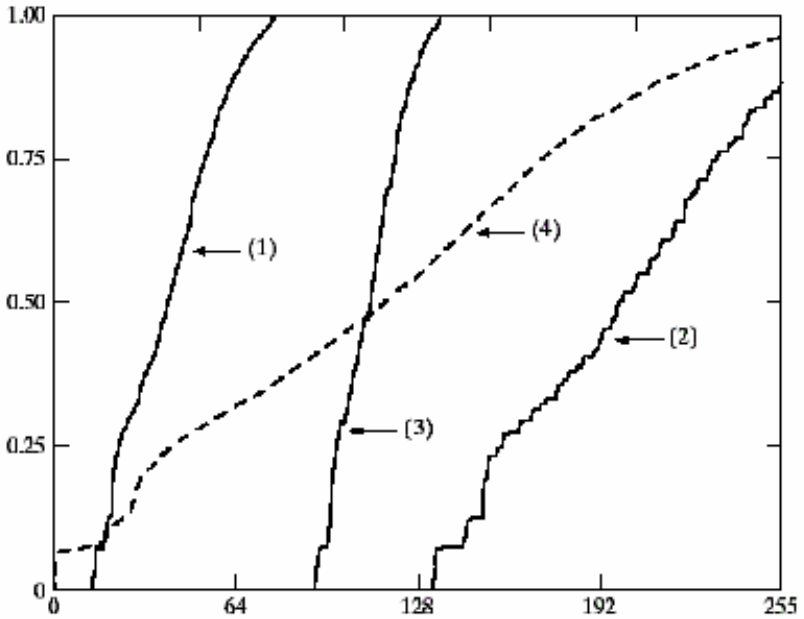


Histogram Equalization



Histogram Equalization





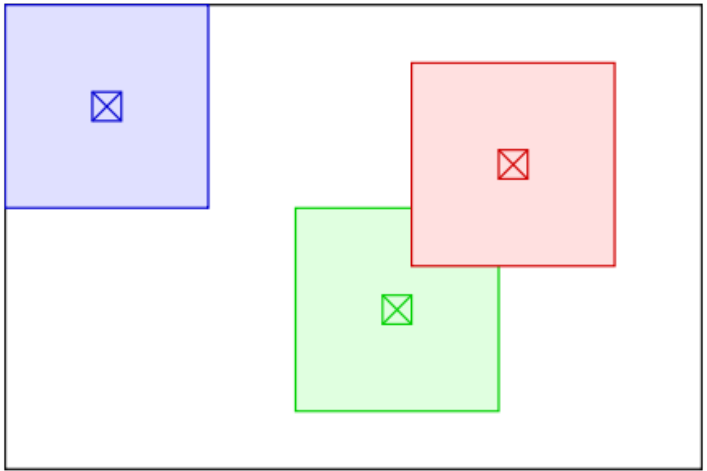
Local Histogram Processing

Local Histogram Processing

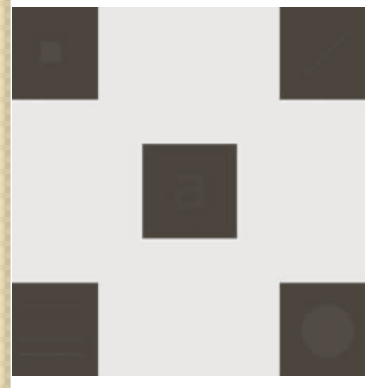
- The previous method was global because pixels are modified by a transformation function based on the intensity distribution of an entire image.
- In some cases, it is necessary to enhance details over small areas in an image.
- The solution is to devise transformation functions based on the intensity distribution in a neighborhood of every pixel in the image.

Local Histogram Processing

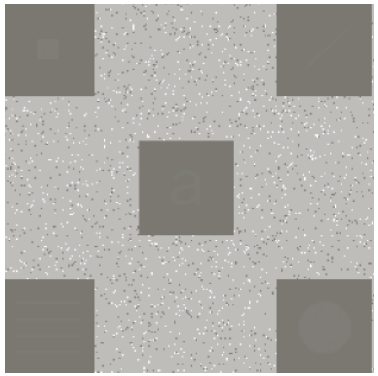
- Define a neighborhood and move its center from pixel to pixel.
- At each location, the histogram of the points in the neighborhood is computed.
- Histogram equalization is applied.



Local Histogram Processing



original



Equalized with global histogram

Equalized with 3X3 local histogram

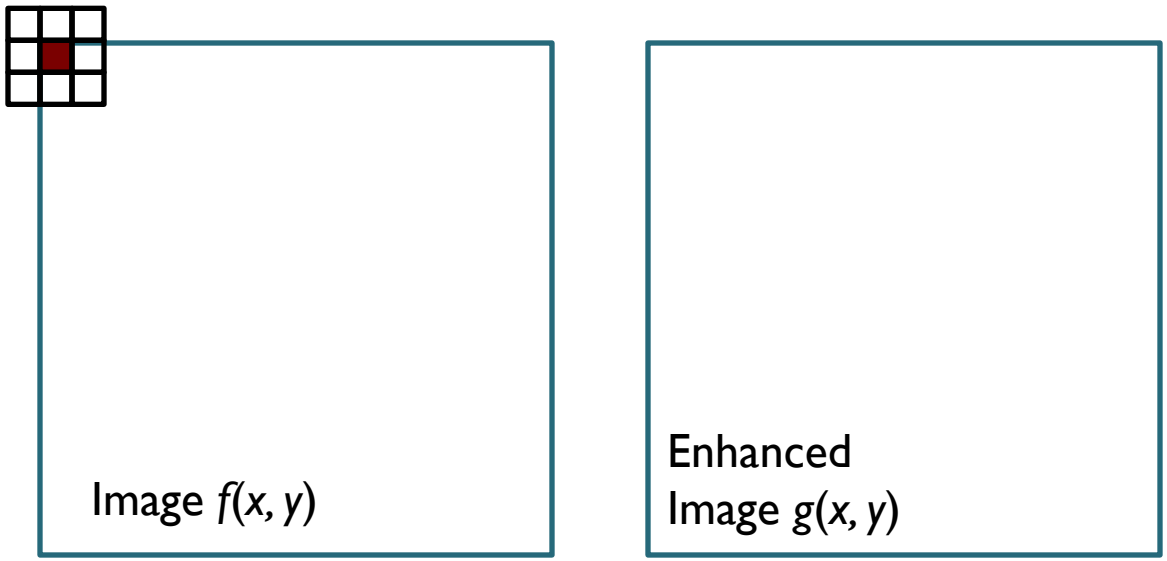
Local Histogram Processing

- Global histogram equalization increases noise
- Local histogram equalization provides more fine details than global histogram equalization
- The intensity values of the objects were too close to the intensity of the large squares and their sizes were too small to influence global histogram equalization.

Spatial Filtering

Spatial Filtering

- Neighborhood (sub-image) rolls over the entire image, each time centering a different pixel.
- For any specific location (x, y), the value of the output image g at those coordinates is equal to the result of applying T to the neighborhood with origin at (x, y).



$$g(x,y) = T[f(x,y)]$$

Spatial Filtering

- This procedure is called Spatial Filtering.
- The neighborhood along with a predefined operation is called a spatial filter.
- Spatial filter is also known as spatial mask, kernel, template or window. Spatial mask is the most popular.

Spatial Filtering

- Spatial filter consists of
  - A neighborhood
  - A predefined operation
- Filtering creates a new pixel with coordinates equal to the coordinates of the center of the neighborhood and whose value is the result of the filtering operation.

Spatial Filtering

- **Linear spatial filter**

  If the operation performed on the image pixels is linear, then the filter is called a linear spatial filter.

- **Non-linear spatial filter**

  If the operation performed on the image pixels is non-linear, then the filter is called a non-linear spatial filter.

Spatial Filtering

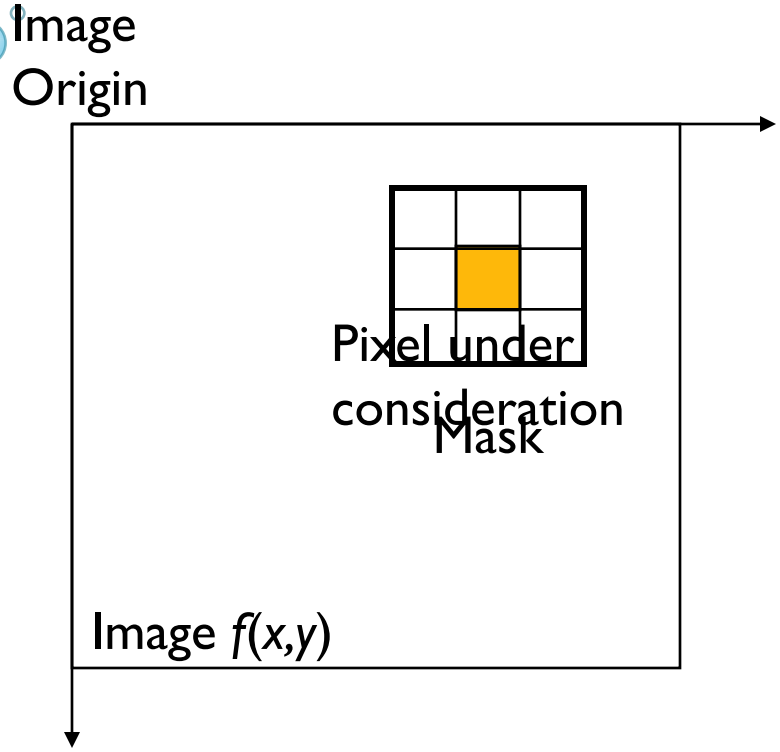
- A *mask* or *filter* or *template* or *kernel* or *window* defines the neighborhood
- Mask size is usually $m \times n$

  $m = 2a + 1, n = 2b + 1$

  where a & b are positive integers.

Let's take a look how it works!!!

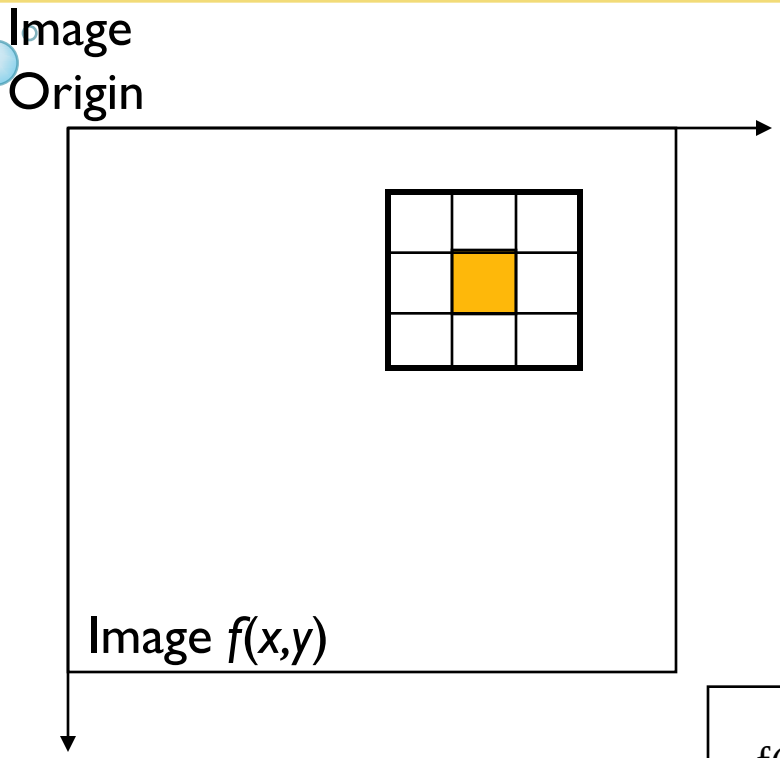
Spatial Filtering



w(-1,-1)	w(-1,0)	w(-1,1)
w(0,-1)	w(0,0)	w(0,1)
w(1,-1)	w(1,0)	w(1,1)

Mask Coefficients showing coordinate arrangement

Spatial Filtering



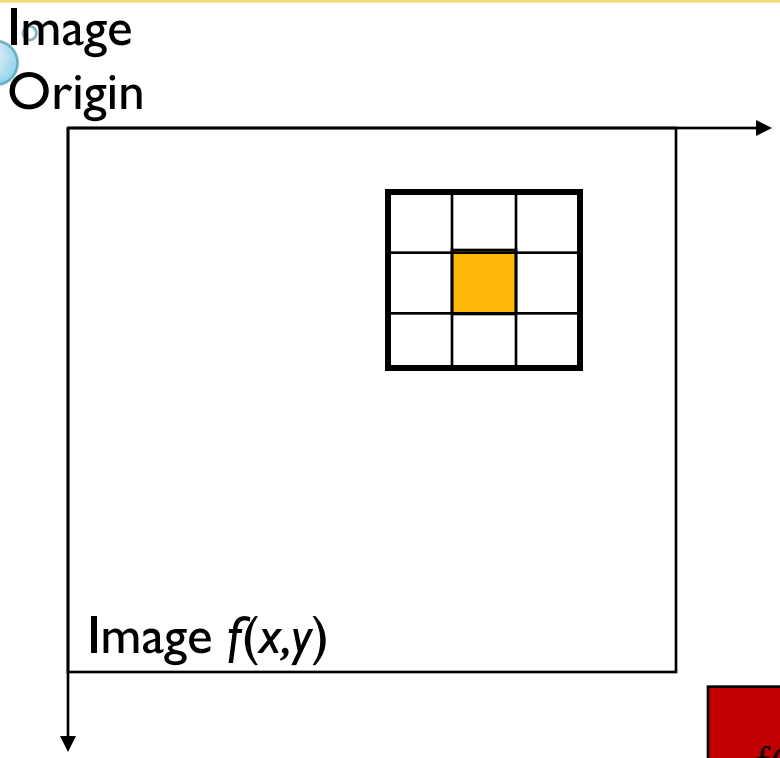
w(-1,-1)	w(-1,0)	w(-1,1)
w(0,-1)	w(0,0)	w(0,1)
w(1,-1)	w(1,0)	w(1,1)

Mask Coefficients showing coordinate arrangement

f(x-1,y-1)	f(x-1,y)	f(x-1,y+1)
f(x,y-1)	f(x,y)	f(x,y+1)
f(x+1,y-1)	f(x+1,y)	f(x+1,y+1)

Pixels of image section under Mask

Spatial Filtering



w(-1,-1)	w(-1,0)	w(-1,1)
w(0,-1)	w(0,0)	w(0,1)
w(1,-1)	w(1,0)	w(1,1)

Mask Coefficients showing coordinate arrangement

f(x-1,y-1)	f(x-1,y)	f(x-1,y+1)
f(x,y-1)	f(x,y)	f(x,y+1)
f(x+1,y-1)	f(x+1,y)	f(x+1,y+1)

Pixels of image section under Mask

Spatial Filtering

Response of the filter at point (x, y):

$$R = w(-1,-1)\, f(x-1, y-1)$$
$$+ w(-1,0)\, f(x-1, y) + \cdots$$
$$\cdots + w(1,0)\, f(x+1, y)$$
$$+ w(1,1)\, f(x+1, y+1)$$

w(-1,-1)	w(-1,0)	w(-1,1)
w(0,-1)	w(0,0)	w(0,1)
w(1,-1)	w(1,0)	w(1,1)

Mask Coefficients showing coordinate arrangement

f(x-1,y-1)	f(x-1,y)	f(x-1,y+1)
f(x,y-1)	f(x,y)	f(x,y+1)
f(x+1,y-1)	f(x+1,y)	f(x+1,y+1)

Pixels of image section under Mask

Spatial Filtering

A more general equation for response:

$$g(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t)\, f(x+s, y+t)$$

Convolution operation

Convolution mask

w(-1,-1)	w(-1,0)	w(-1,1)
w(0,-1)	w(0,0)	w(0,1)
w(1,-1)	w(1,0)	w(1,1)

Spatial Correlation and Convolution

Spatial Correlation

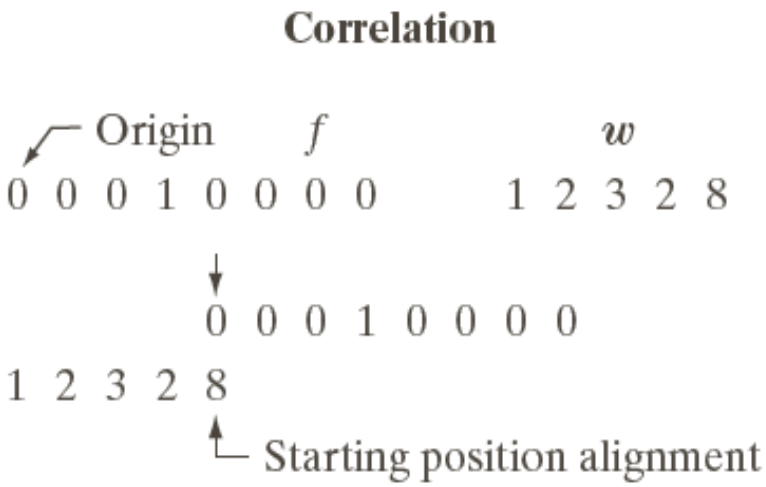
0 0 0 1 0 0 0 0

Discrete unit impulse function

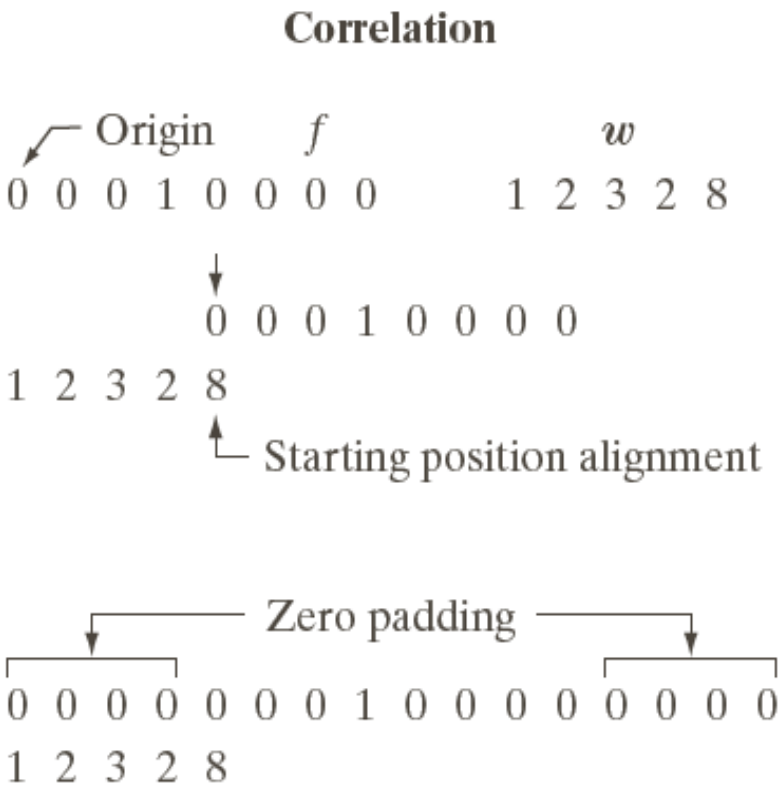
Spatial Correlation

Correlation

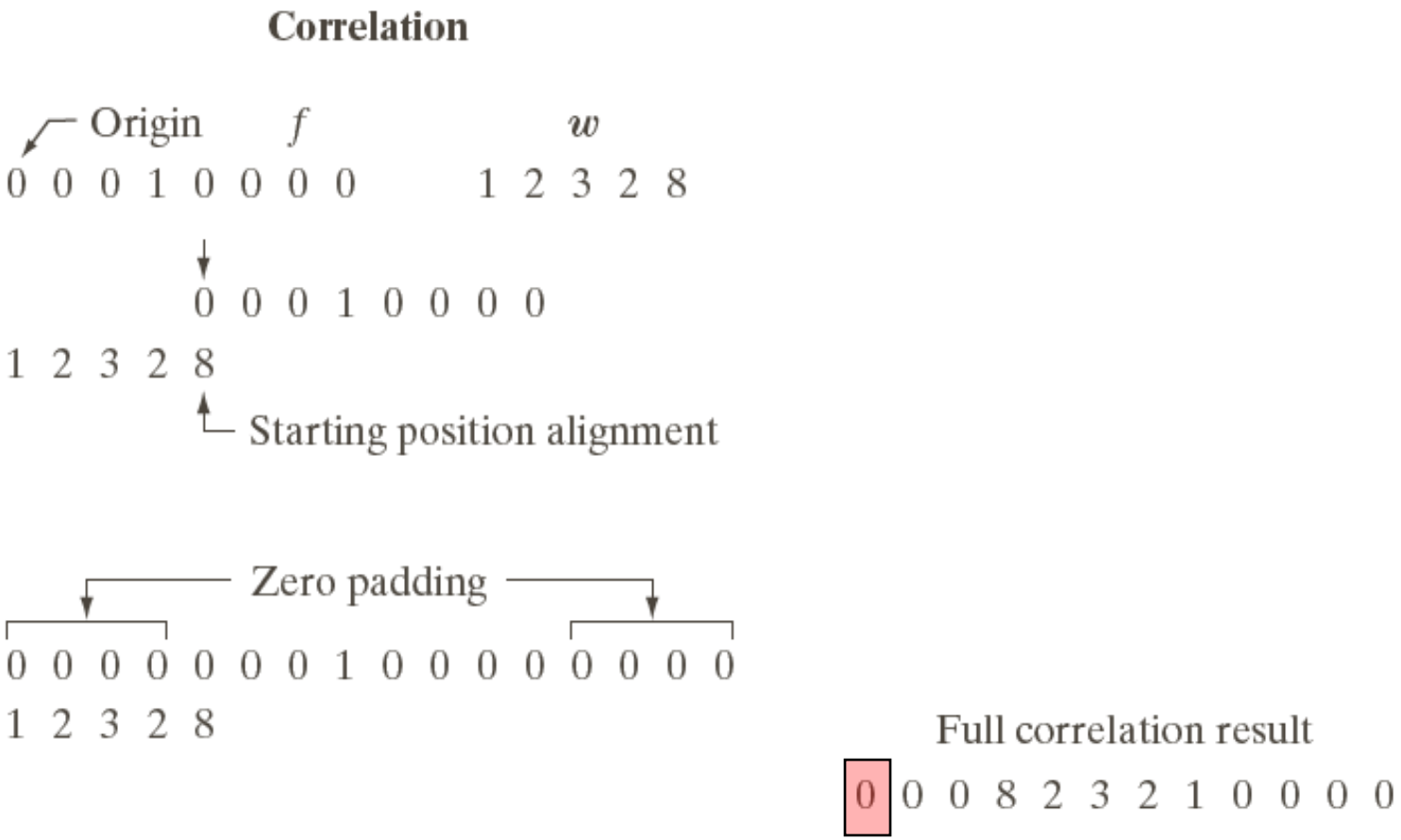
Spatial Correlation



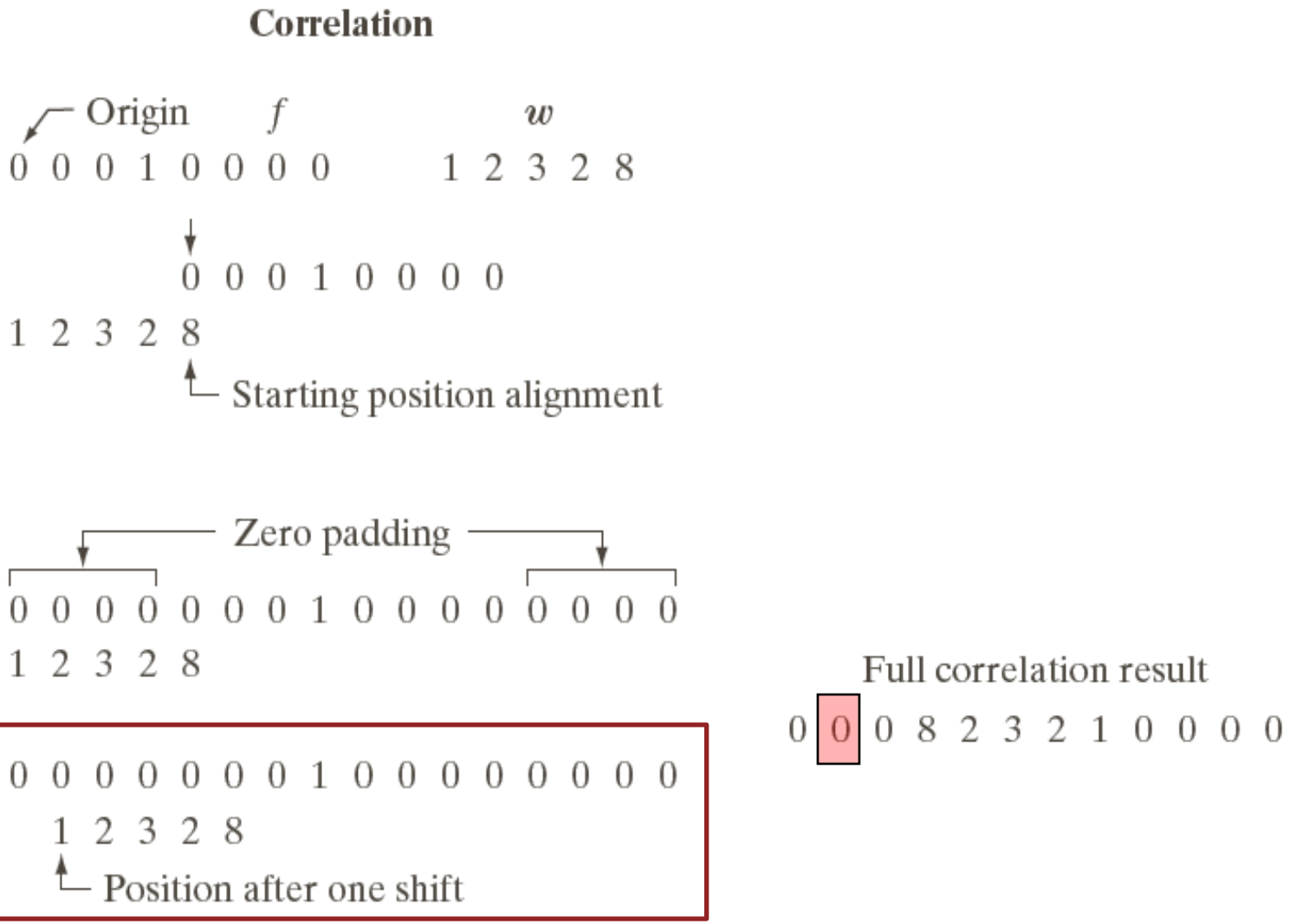
Spatial Correlation



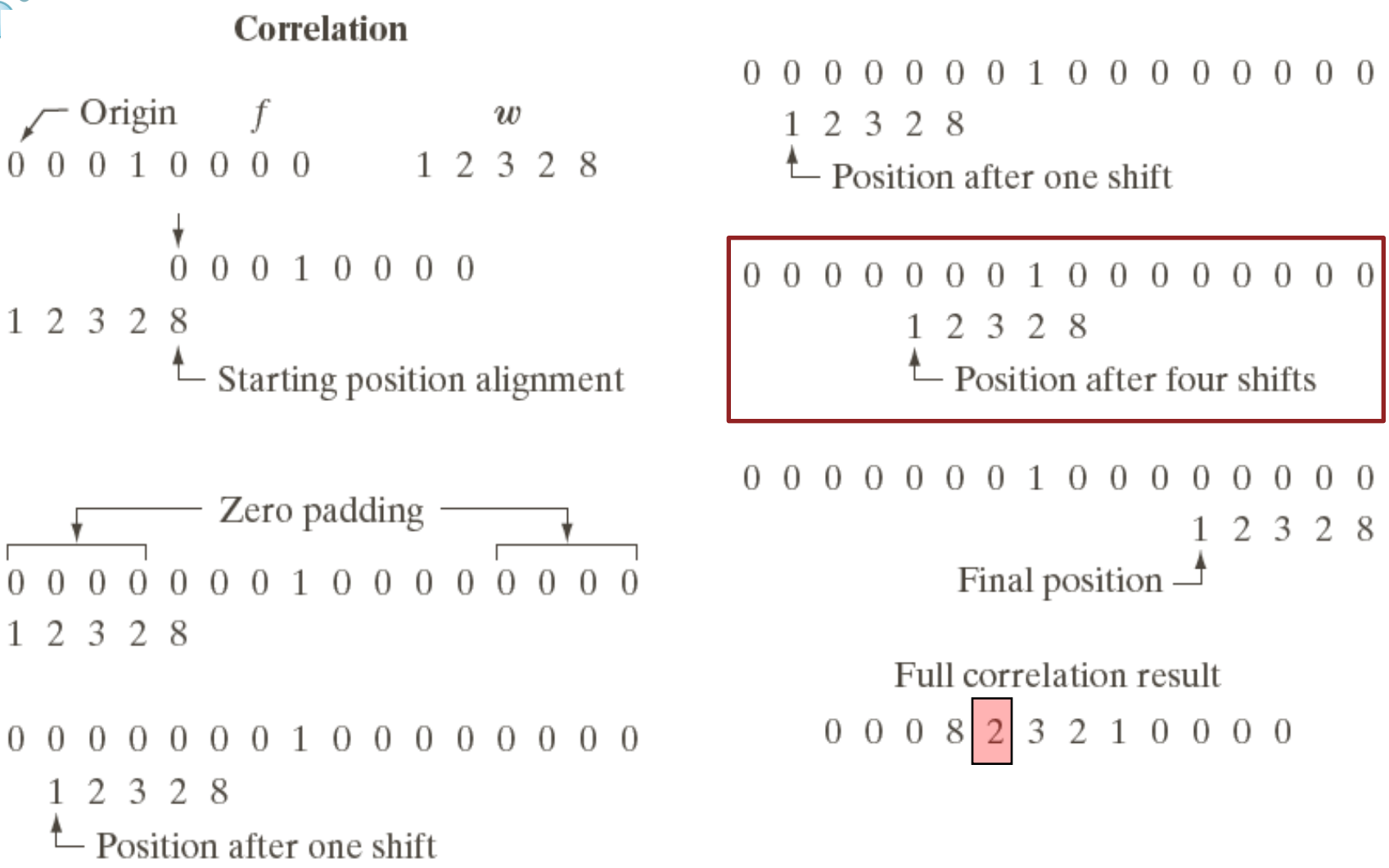
Spatial Correlation



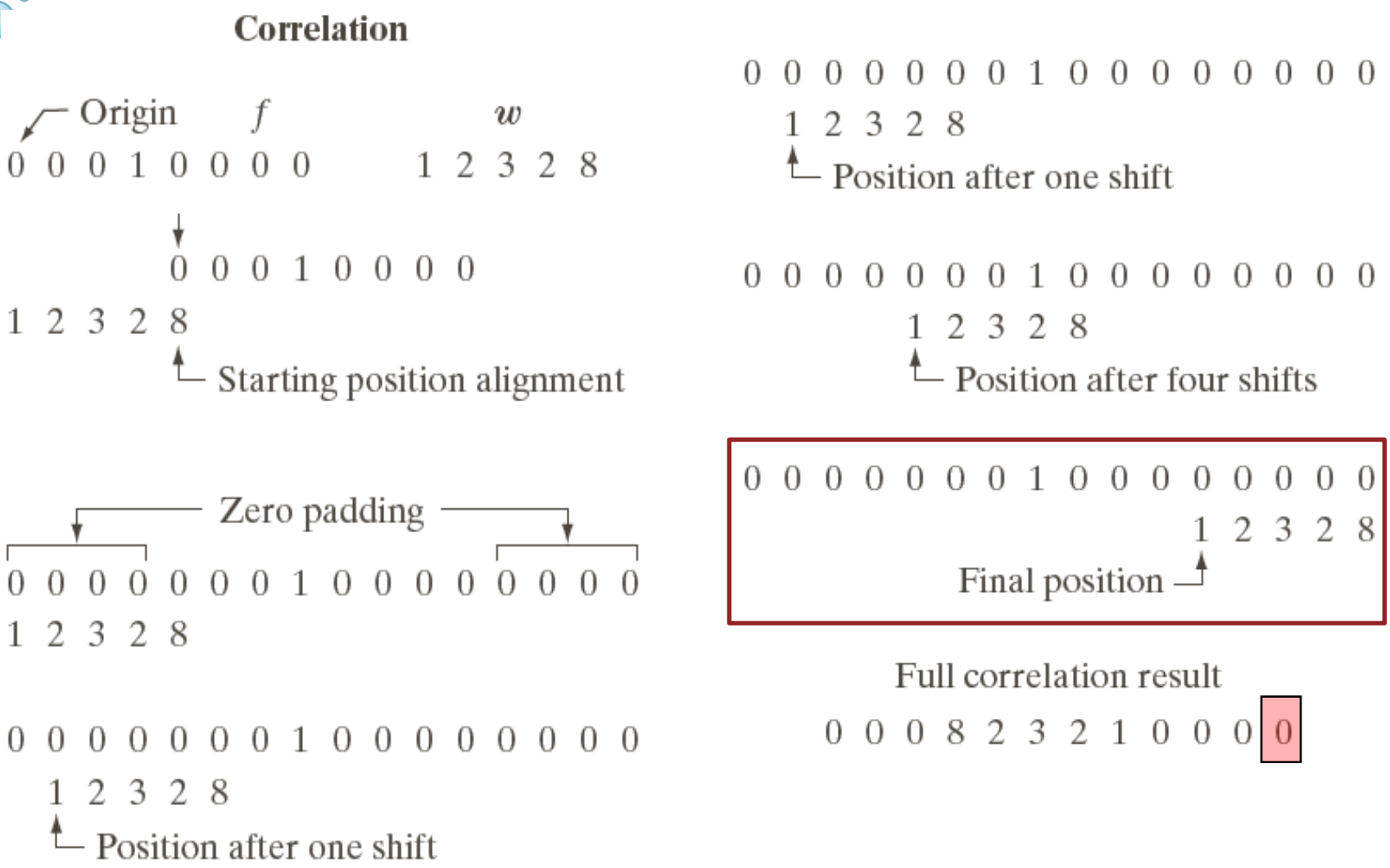
Spatial Correlation



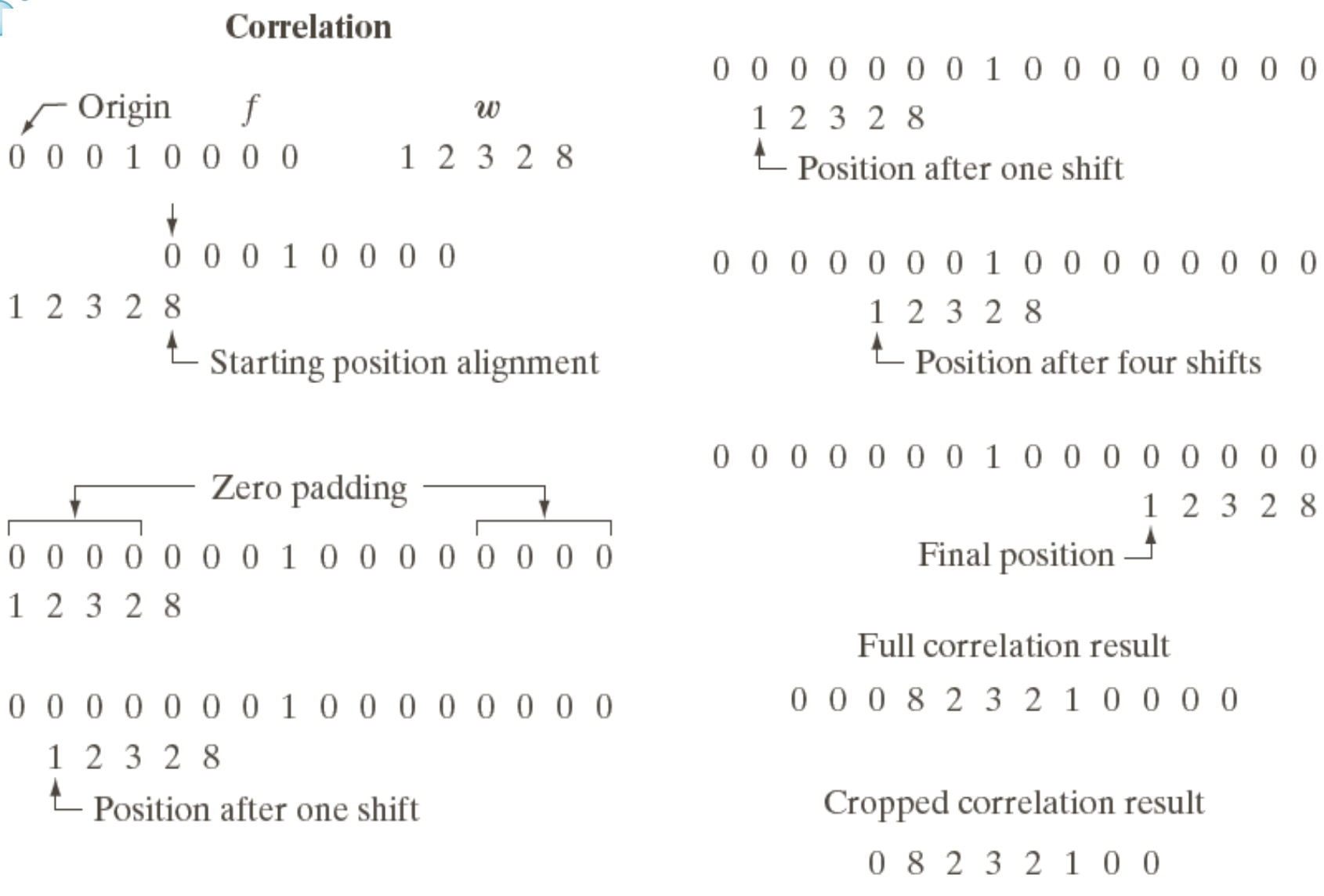
Spatial Correlation



Spatial Correlation



Spatial Correlation



Spatial Convolution

Now, Convolute 0 0 0 1 0 0 0 0 by 1 2 3 2 8

Steps

- Reverse 1 2 3 2 8 to 8 2 3 2 1
- Then apply correlation as mentioned in previous slides

Spatial Convolution

Convolution

Origin f w rotated 180°

0 0 0 1 0 0 0 0 8 2 3 2 1

0 0 0 1 0 0 0 0

8 2 3 2 1 Starting position

0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0

8 2 3 2 1 Full padded

0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0

8 2 3 2 1 Position after one shift

0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0

8 2 3 2 1 Position after one shift

0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0

8 2 3 2 1 Position after 4 shifts

0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0

Final position 8 2 3 2 1

Full convolution result

0 0 0 1 2 3 2 8 0 0 0 0

Cropped convolution result

0 1 2 3 2 8 0 0

2D Correlation and Convolution

240	225	210	190	160
220	195	170	140	115
190	160	145	125	100
150	130	115	95	75
110	85	65	50	40
70	60	45	30	15

An Image

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

Mask

Apply the mask and calculate the first row

Issues in spatial filtering

- When the mask moves closer to the border
  - Closer to that distance: some rows/columns of the mask are out of image
- Way out:
  - Ignoring the outside columns/rows
  - Padding
    - Zero padding
    - Mirror padding

Smoothing Spatial Filters

Smoothing Spatial Filters

- Smoothing filters are used for
  - blurring
  - noise reduction
  - sharp transition reduction
  - finding large objects, ignoring small details

Smoothing Linear Filters

- The output of a smoothing linear spatial filter is simply the (weighted) average of the pixels contained in the neighborhood.
- Also known as averaging filters or lowpass filters.

Smoothing Linear Filters

$$R = \frac{1}{9}\sum_{i=1}^{9} z_i$$

$$\frac{1}{9} \times \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array}$$

A spatial averaging filter in which all coefficients are equal is called a box filter.

$$R = \frac{1}{16}\sum_{i=1}^{16} w_i z_i$$

$$\frac{1}{16} \times \begin{array}{|c|c|c|} \hline 1 & 2 & 1 \\ \hline 2 & 4 & 2 \\ \hline 1 & 2 & 1 \\ \hline \end{array}$$

This mask yields a weighted average. It gives more importance to the center pixels and the 4-neighbors.

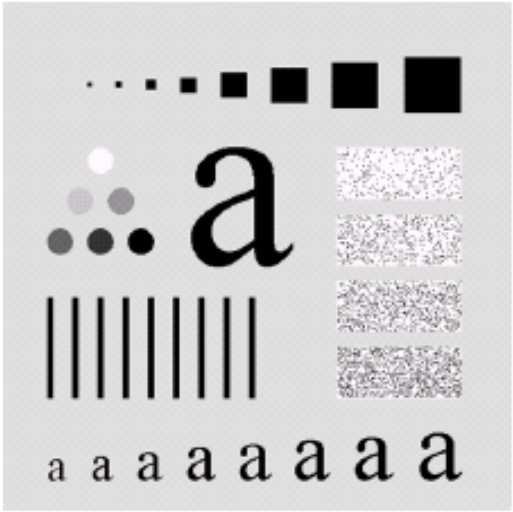
Smoothing Linear Filters

```
clc;
clear all;
close all;
I = imread('cameraman.tif');
figure, imshow(I);

h = fspecial('average',5); % h = fspecial('average',hsize)
localMean = imfilter(I,h,'replicate');
imshowpair(I,localMean,'montage')
```

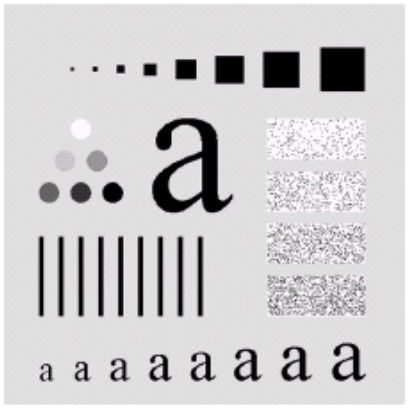
Smoothing Linear Filters

- Black square: 3, 5, 9, 15, 25, 35, 45
- Border 25 pixels apart
- Circle: 25 pixels, gray: 0, 20, ..., 100%
- Small a's: 10, 12, 24 pixels
- Large a: 60 pixels
- Bars: 5X100 pixels, Sep: 20 pix
- Noise boxes: 50X120 pixels



Smoothing Linear Filters

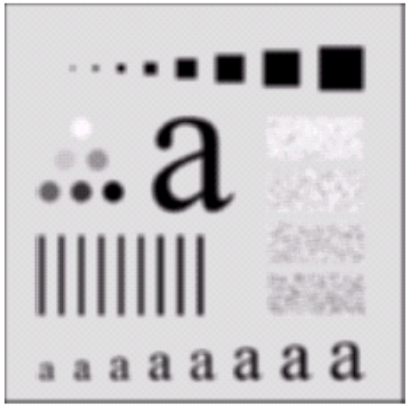
Original Image



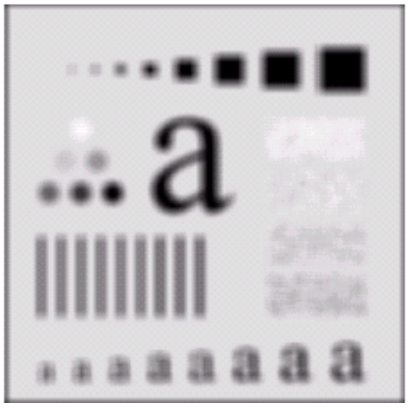
After applying
3 × 3 filter

After applying
5 × 5 filter

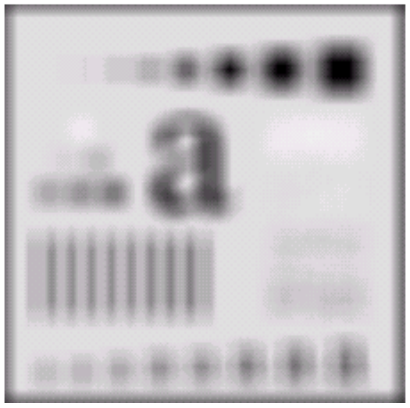
After applying
9 × 9 filter



After applying
15 × 15 filter

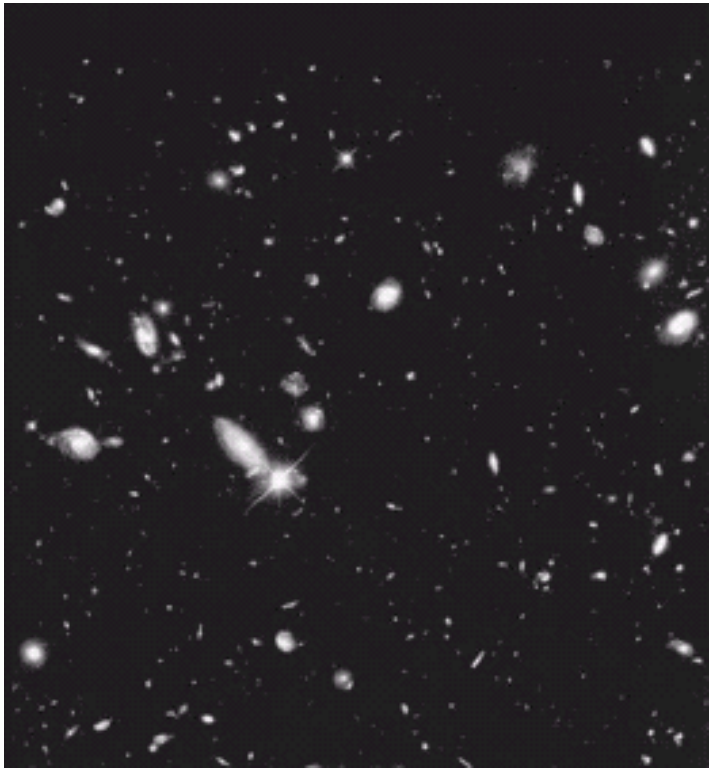


After applying
25 × 25 filter

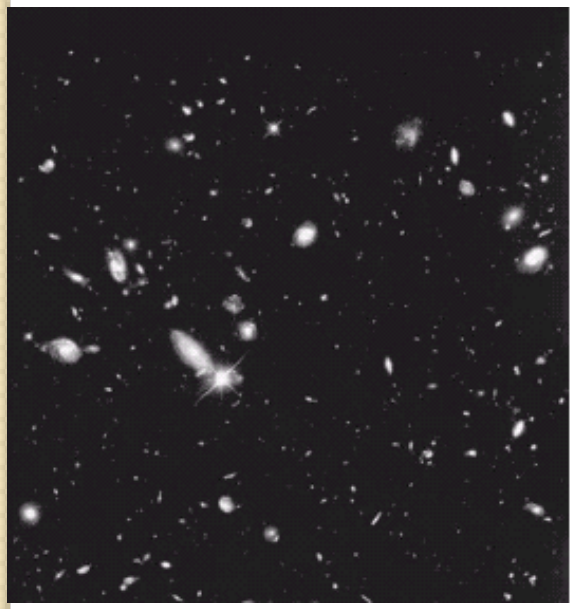


Smoothing Linear Filters

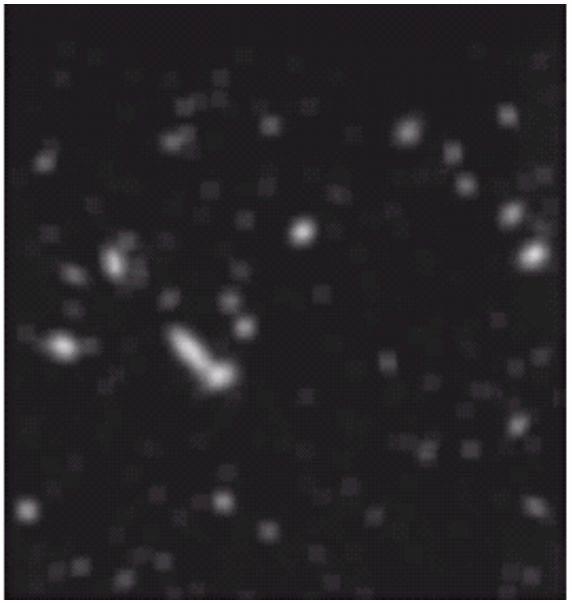
- Blurring helps to find major objects
- Removes fine details
- Original image contains lots of small objects



Smoothing Linear Filters



Original image



Smoothed by
15X15 mask

Thresholding
after smoothing

Smoothing Non-linear Filters

Order-Statistic(Nonlinear) Filters

- Nonlinear filtering
- Response is based on ordering(ranking) the pixels
- Example : Median filter

Median Filter

Advantage

- For certain type of random noise, they provide excellent noise reduction
- Considerably less blurring effect
- Particularly effective in the presence of impulse noise, also called salt-and-pepper noise.

Median Filter

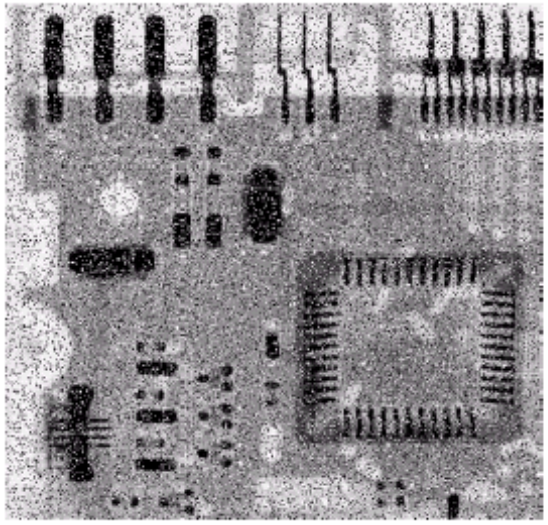
- Policy:
  - Sort the values of enclosed pixels
  - Select the median as output pixel level
- Example:
  - Let, 3X3 mask size

10	20	20
20	100	20
25	20	15

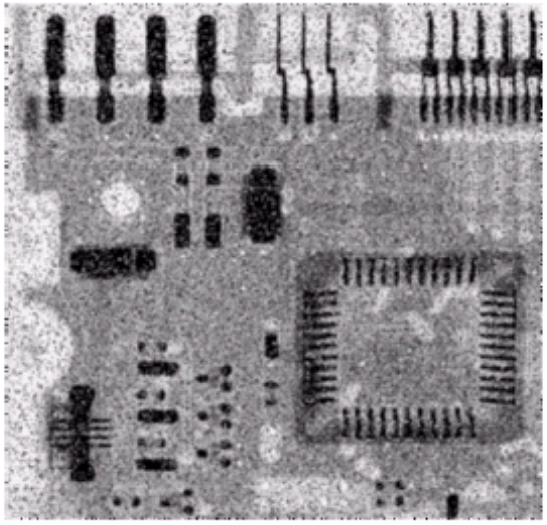
Image pixel values under masks

  - Sorted values: 10, 15, 20, 20, 20, 20, 20, 25, 100
  - Median is 20
  - Output pixel value is 20

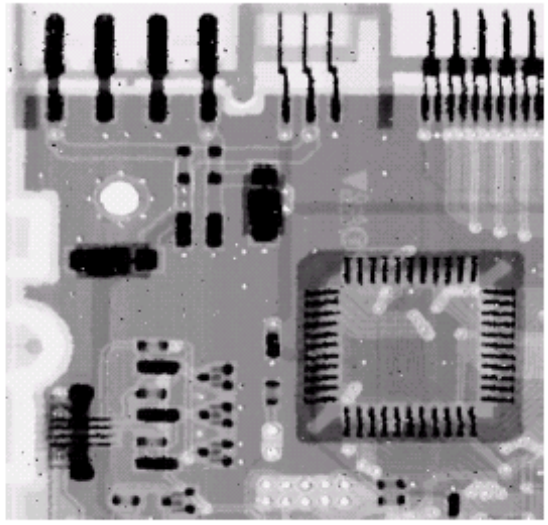
Median Filter



Noisy image



Noise reduction
by Avg. filter



Noise reduction
by median filter

Histogram processing

The **histogram** of a digital image with **L** total possible intensity levels in the range **[0,G]** is defined as the discrete function:

$$h(r_k) = n_k$$

Where

- ✓ r_k is the k_{th} intensity level in the interval **[0,G]**
- ✓ n_k is the number of pixels in the image whose intensity level is r_k
- ✓ **G**: [*255* for images of class *uint8*, *65535* for images class *uint16* and *1.0* for images of class *double*].

Histogram processing

Normalized histograms: can be obtained by dividing all elements of $h(r_k)$ by the total number of pixels in the image:

$$P(r_k) = \frac{h(r_k)}{n} = \frac{n_k}{n} \quad \text{, for } k = 1, 2, \ldots, L$$

n total number of pixels

From probability:

- $P(r_k)$ – *Estimation* of the probability of occurrence of intensity level r_k. (The sum of all components of a normalized histogram is equal =*1*).

Histogram processing

Histogram Equalization

- **Histogram Equalization:** is a method which increases the dynamic range of the gray-level in a low-contrast image to cover full range of gray-levels.
- ***Histogram equalization*** is achieved by having a transformation function $T(r)$, which can be defined to be the **Cumulative Distribution Function (CDF)** of a given **Probability Density Function (PDF)** of a gray-levels in a given image (the histogram of an image can be considered as the ***approximation*** of the **PDF** of that image).

Histogram processing

Continuous Case

- For intensity levels that are continuous quantities ***normalised*** to the range ***[0, 1].***

Let $P_r(r)$ – probability density function (**PDF**) of the intensity levels. The following transformation on the input levels to obtain output levels, s:

$$S = T(r) = \int_0^r P_r(\omega) d\omega$$

ω -***dummy*** variable of integration.

- The **PDF** of the output levels is uniform:

$$P_s(s) = \begin{cases} 1 & 0 \ll s \ll 1 \\ 0 & otherwise \end{cases}$$

- Image, whose intensity levels are equally likely, and it cover the entire range ***[0 ,1].***

- This transformation is called **intensity-levels equalization process** (and it's nothing more than the cumulative distribution function (**CDF**)).

Discrete case (quantities):

- The equalization transformation becomes:

$$S_k = T(r_k) = \sum_{j=1}^{k} P_r(r_j) = \sum_{j=1}^{k} \frac{n_j}{n}, \quad for\ k = 1, 2, \ldots, L,$$

S_k – Intensity value of the output image corresponding to value r_k in the input image.

Histogram processing

- As noted earlier, the transformation function $T(r_k)$ is simply the ***cumulative sum of normalized histogram*** values, we can use the function **cumsum** to obtain the transformation function, type:

```
>> hnorm = imhist (f)./numel(f);
>> cdf = cumsum (hnorm);
```

- A plot of **CDF** (for example) can be obtained using the following commands:

```
>> x= linspace (0, 1, 256);
% Intervals for [0, 1] horiz scale
>> plot (x, cdf) % plot cdf vs. x
>> axis ([0   1   0   1]) % scale, setting and labels
>> set (gca, 'xtick', 0:0.2:1)
>> set (gca, 'ytick', 0:0.2:1)
>> xlabel ('Input Intensity values', 'fontsize' ,9)
>> ylabel ('Output Intensity values', 'fontsize', 9)
>> %specify text in the body of the graph:
>> text (0.18, 0.5,'Transformation function',...
        'fontsize', 9)
```

Histogram processing

CDF Manual Calculation

Example :

- Consider an 8-bit gray scale image has the following values:

$$\begin{bmatrix} 52 & 55 & 61 & 66 & 70 & 61 & 64 & 73 \\ 63 & 59 & 55 & 90 & 109 & 85 & 69 & 72 \\ 62 & 59 & 68 & 113 & 144 & 104 & 66 & 73 \\ 63 & 58 & 71 & 122 & 154 & 106 & 70 & 69 \\ 67 & 61 & 68 & 104 & 126 & 88 & 68 & 70 \\ 79 & 65 & 60 & 70 & 77 & 68 & 58 & 75 \\ 85 & 71 & 64 & 59 & 55 & 61 & 65 & 83 \\ 87 & 79 & 69 & 68 & 65 & 76 & 78 & 94 \end{bmatrix}$$

8*8 sub image

- The ***histogram*** for this sub image is shown in the following table. Pixels values that have a zero count are ***excluded***.

Histogram processing

Value	Count	Value	Count	Value	Count	Value	Count
52	1	66	2	77	1	106	1
55	3	67	1	78	1	109	1
58	2	68	5	79	2	113	1
59	3	69	3	83	1	122	1
60	1	70	4	85	2	126	1
61	4	71	2	87	1	144	1
62	1	72	1	88	1	154	1
63	2	73	2	90	1		
64	2	75	1	94	1		
65	3	76	1	104	2		

Histogram processing

- The *cumulative distribution function* (*cdf*) is shown below:

Value	cdf	Value	cdf	Value	cdf	Value	cdf	Value	cdf
52	1	64	19	72	40	85	51	113	60
55	4	65	22	73	42	87	52	122	61
58	6	66	24	75	43	88	53	126	62
59	9	67	25	76	44	90	54	144	63
60	10	68	30	77	45	94	55	154	64
61	14	69	33	78	46	104	57		
62	15	70	37	79	48	106	58		
63	17	71	39	83	49	109	59		

Histogram processing

- This *cdf* table shows that the minimum value in the sub image is **52** and the maximum value is **154**.

$$cdf(52) = 1, \qquad cdf(154) = 64$$

- The *cdf* must be normalized to **[0, 255]**,

  - The **general histogram equalization formula** is:

$$h(v) = round\left(\frac{cdf(v) - cdf_{min}}{(M * N) - cdf_{min}} * (L - 1)\right)$$

Where:

- cdf_{min}- The minimum value of the *cdf*
- $M * N$: image's number of pixels. (***M-width, N-height***)
- L -number of gray scale levels (in most cases, **255**).
- The equalization formula for this particular example, is

$$h(v) = round\left(\frac{cdf(v) - 1}{63} * 255\right)$$

Histogram processing

- For example: the $cdf(78) = 46$, the *normalized* value becomes:

$$h(78) = round\left(\frac{46-1}{63} * 255\right) = round\,(0.714286 * 255) = 182$$

- We use the above formula to calculate the normalized *cdf* and the values of the equalized image are directly taken from normalized *cdf*, the following table contains the normalized *cdf*.

0	12	53	93	146	53	73	166
65	32	12	215	235	202	130	158
57	32	117	239	251	227	93	166
65	20	154	243	255	231	146	130
97	53	117	227	247	210	117	146
190	85	36	146	178	117	20	170
202	154	73	32	12	53	85	194
206	190	130	117	85	174	182	219

Sharpening Spatial Filters

The concept of sharpening filter

Sharpening Spatial Filters

To highlight fine detail in an image or to enhance detail that has been blurred, either in error or as a natural effect of a particular method of image acquisition.

Blurring vs Sharpening

•Blurring/smooth is done in spatial domain by pixel averaging in a neighbors, it is a process of integration.

• Sharpening is an inverse process, to find the difference by the neighborhood, done by spatial differentiation. spatial differentiation.

Sharpening Spatial Filters

- Sharpening filters highlights transitions in intensity
- Very useful for highlighting fine details
- Helps to remove blurring

Smoothing vs Sharpening

Smoothing

Averaging

Integration

Sharpening

Difference

Differentiation

Image Sharpening

Sharpening

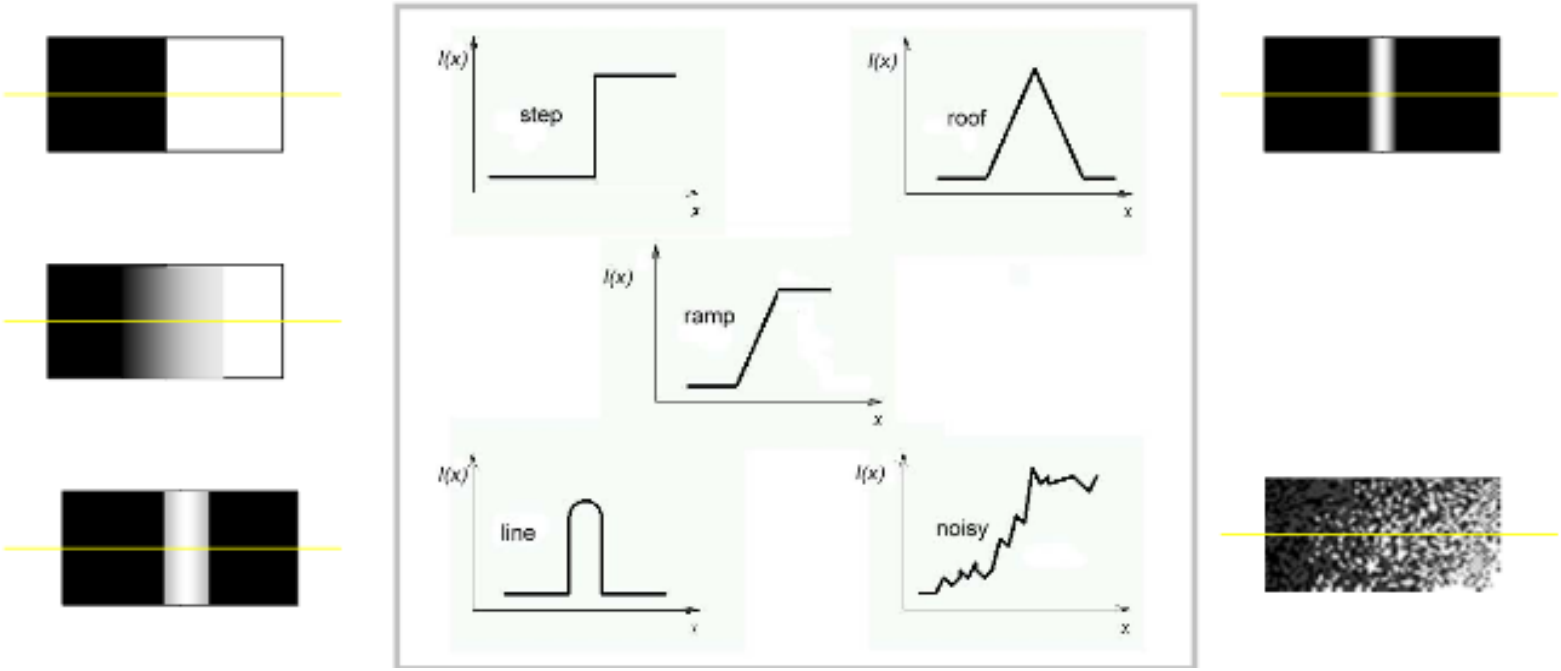
Difference

Differentiation

- Strength of response of derivative proportional to image discontinuity
- Sharp changes noise point, edges, lines, grey ramp are easily detected

Sharpening Spatial Filters

- **Edge is a boundary between two regions with relatively distinct gray level properties.**
- **Edges are pixels where the brightness function changes abruptly.**
- **Edge detectors are a collection of very important local image pre-processing methods used to locate (sharp) changes in the intensity function.**



Criteria for Optimal Edge Detection

(1) Good detection

- Minimize the probability of false positives (i.e., spurious edges).
- Minimize the probability of false negatives (i.e., missing real edges).

(2) Good localization

- Detected edges must be as close as possible to the true edges.

Criteria for Optimal Edge Detection

- Often, points that lie on an edge are detected by:

(1) Detecting the local **maxima** or **minima** of the first derivative.

(2) Detecting the **zero-crossings** of the second derivative.

Criteria for Optimal Edge Detection

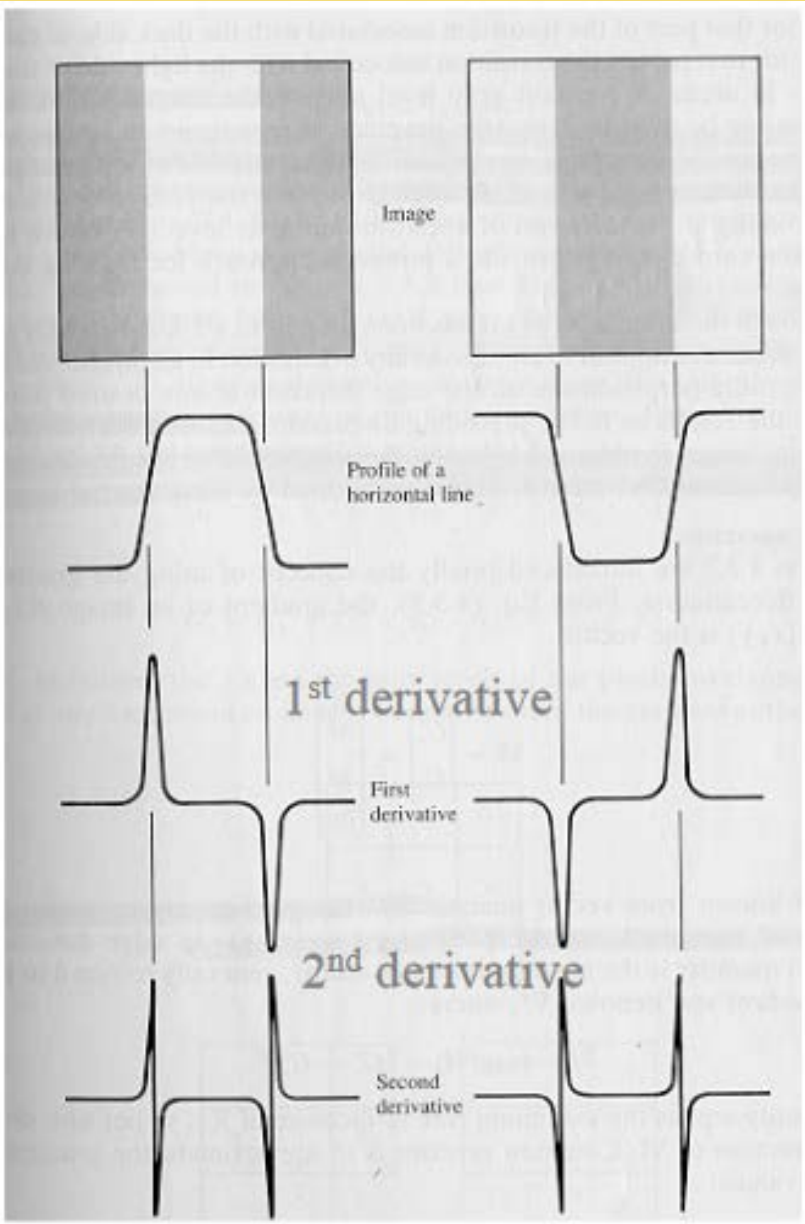


Image Sharpening

Derivative operator

•The strength of the response of a derivative operator is proportional to the degree of operator is proportional to the degree of discontinuity of the image at the point at which the operator is applied.

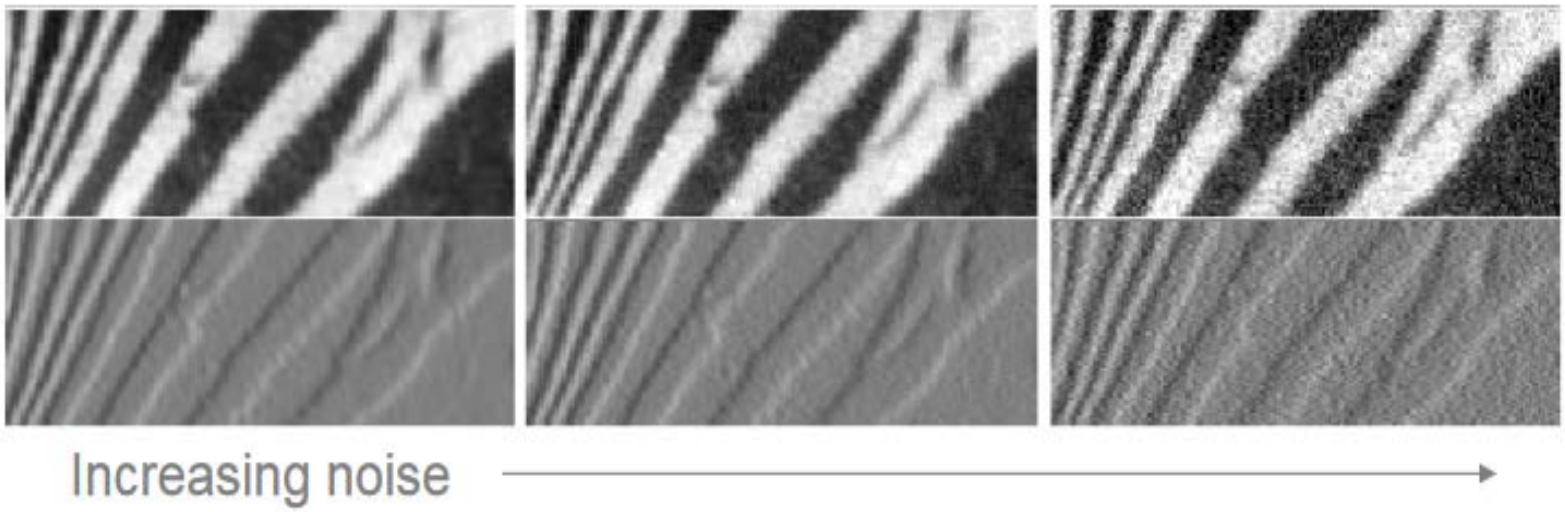
•Image differentiation

–enhances edges and other discontinuities (noise)
–deemphasizes area with slowly varying gray-level values.

Derivatives and Noise

- Derivatives are strongly affected by noise
  - obvious reason: image noise results in pixels that look very different from their neighbors
  - The larger the noise - the stronger the response
- What is to be done?
  - Neighboring pixels look alike
  - Pixel along an edge look alike
  - Image smoothing should help
  - Force pixels different to their neighbors (possibly noise) to look like neighbors

Derivatives and Noise



- Need to perform image smoothing as a preliminary step
- Generally – use Gaussian smoothing

Image Sharpening

First and second order difference of 1D

- The basic definition of the first-order derivative of a one-dimensional function f(x) is the difference

$$\frac{\partial f}{\partial x} = f(x+1) - f(x)$$

- The second-order derivative of a one-dimensional function f(x) is the difference

$$\frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x)$$

Image Sharpening

- 1st and 2nd order derivatives will be used
- Behavior to check
  - Const gray level
  - At onset and end of discontinuities (ramp and step)
  - Along gray-ramp

Properties of Derivatives

- Value of 1st order derivative will be
  - 0 at const gray level
  - Nonzero at onset of step and ramp
  - Nonzero along ramp

- Value of 2nd order derivative will be
  - 0 at const gray level
  - Nonzero at onset and end of step and ramp
  - 0 along ramp

Digital 1st Order Derivative

$$\frac{\partial f}{\partial x} = \frac{\text{change of } f}{\text{change of } x}$$

$$\frac{\partial f}{\partial x} = \frac{f(x+1) - f(x)}{x+1-x}$$

$$= f(x+1) - f(x)$$

Digital 2nd Order Derivative

$$\frac{\partial^2 f}{\partial x^2} = \frac{\text{change of } \frac{\partial f}{\partial x}}{\text{change of } x}$$

$$\frac{\partial^2 f}{\partial x^2} = \frac{f(x+1) - f(x) - (f(x) - f(x-1)}{x+1-x}$$

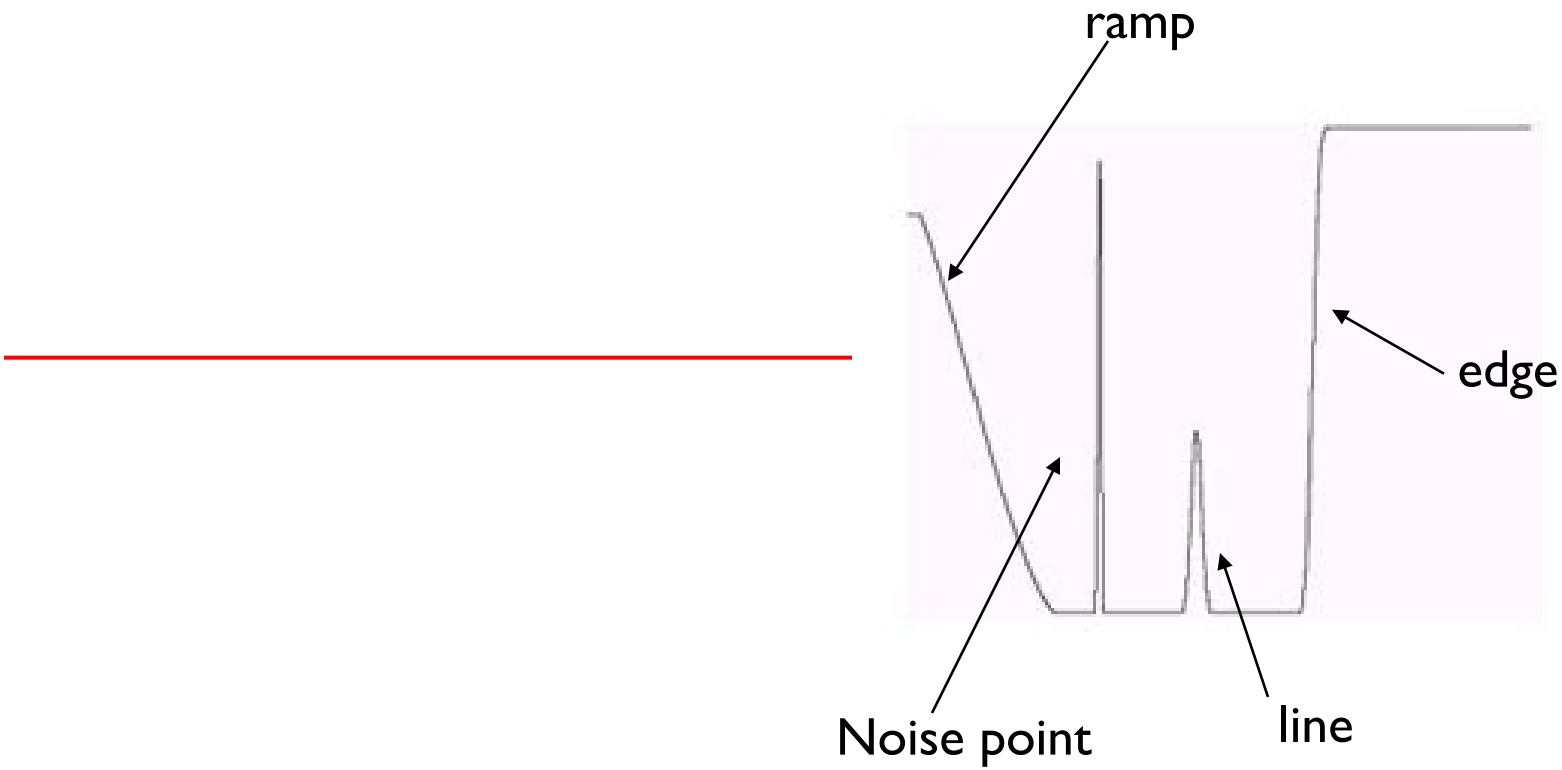
$$= f(x+1) + f(x-1) - 2f(x)$$

1st Order & 2nd Order Derivative

- Gray-ramp (smooth transition betn white and black)
- Isolated noise point
- Line
- Edge

1st Order & 2nd Order Derivative

1st Order & 2nd Order Derivative

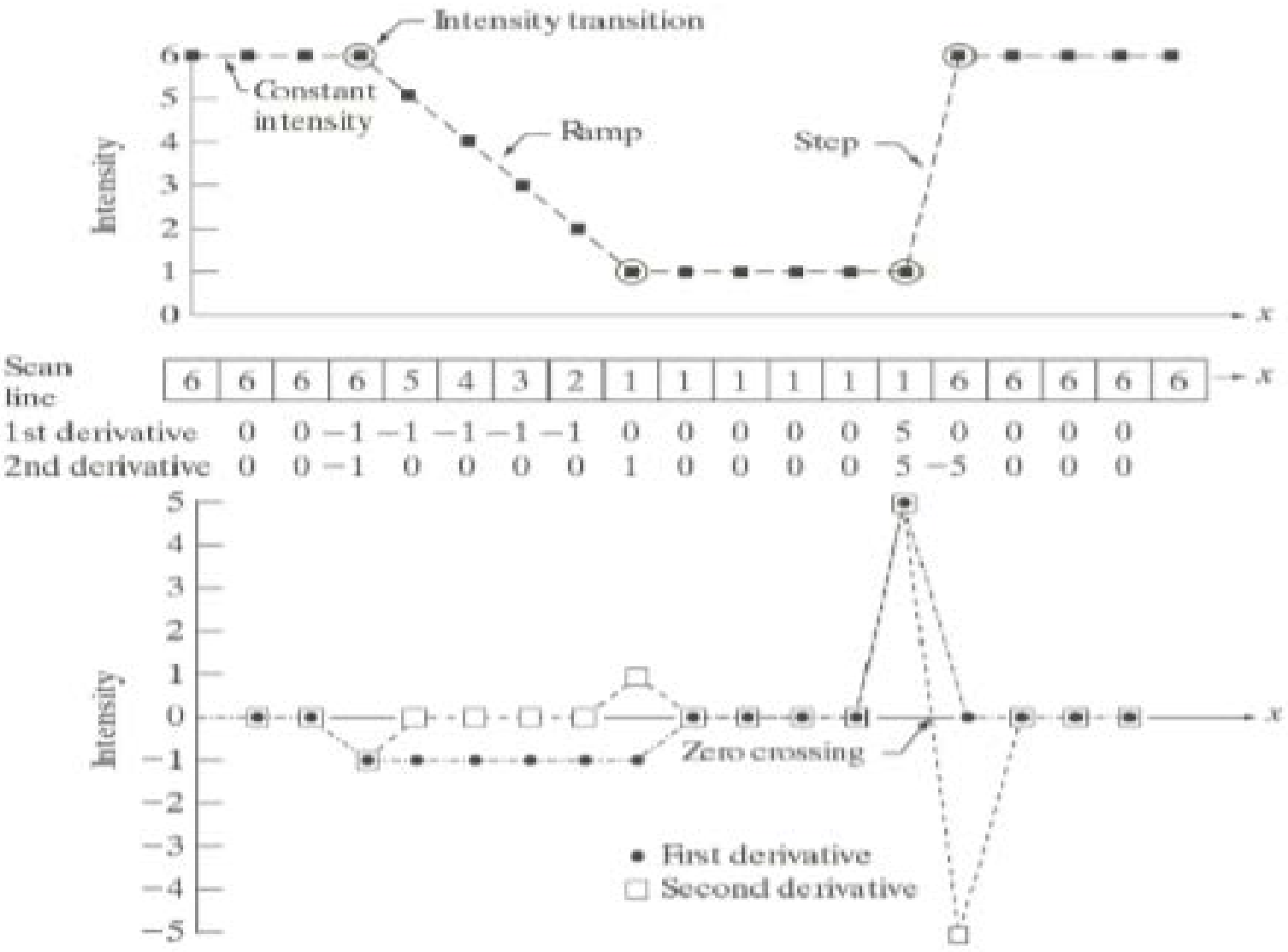


1st Order & 2nd Order Derivative

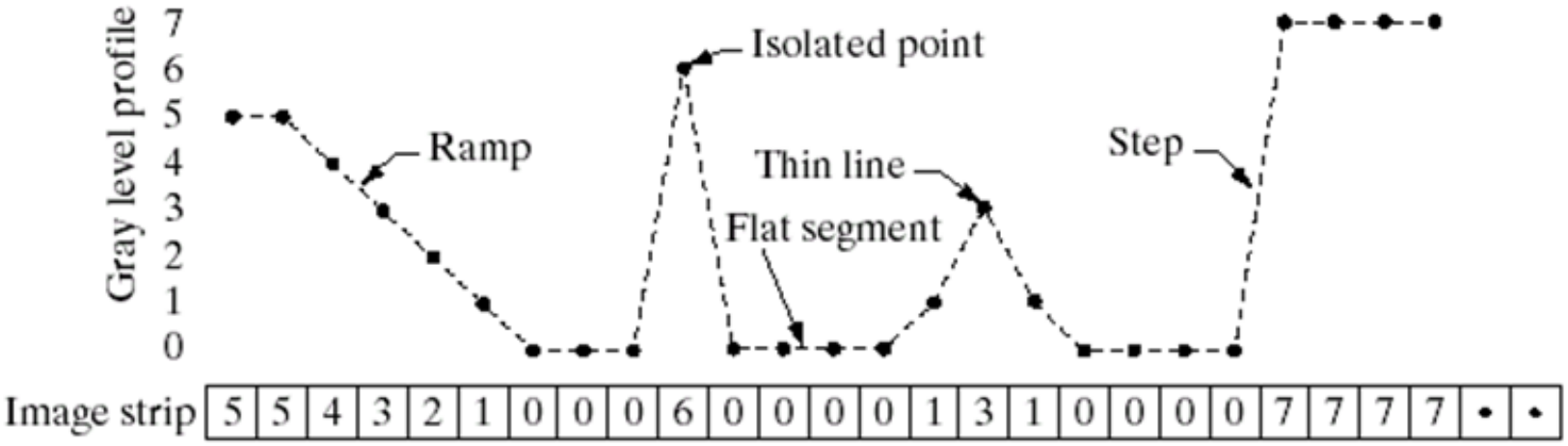
$$\frac{\partial f}{\partial x} = f(x+1) - f(x)$$

$$\frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x)$$

1st Order & 2nd Order Derivative

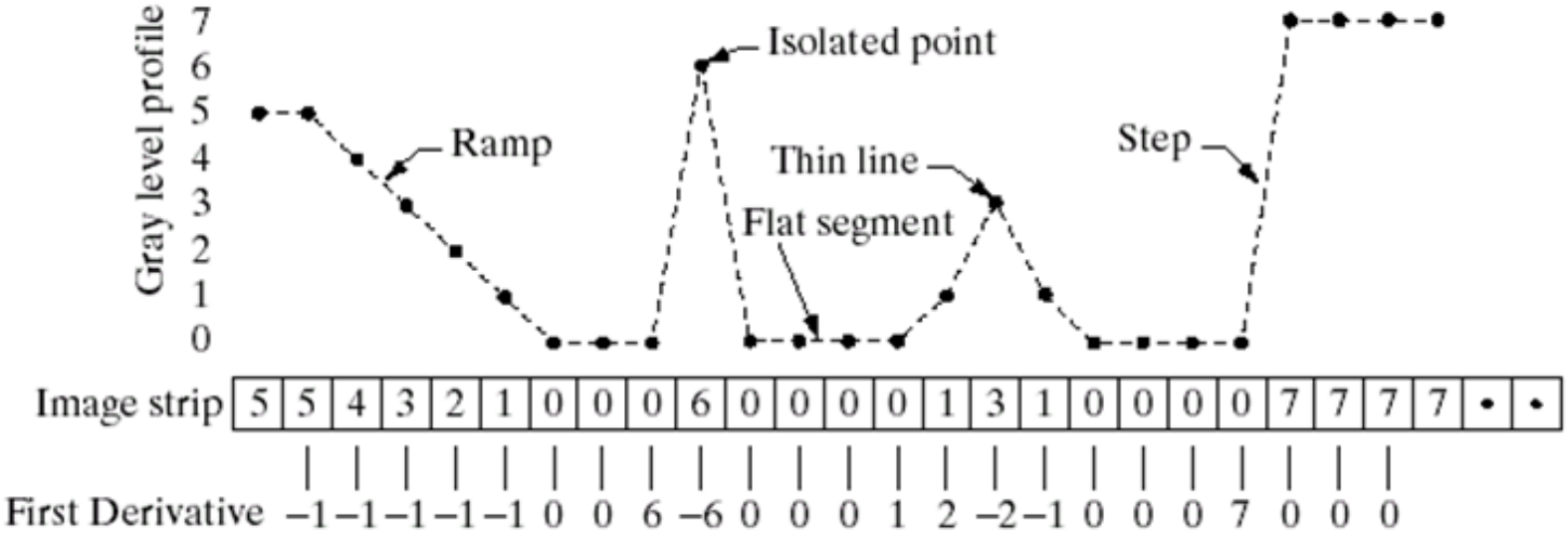


1st Order & 2nd Order Derivative



1st Order & 2nd Order Derivative

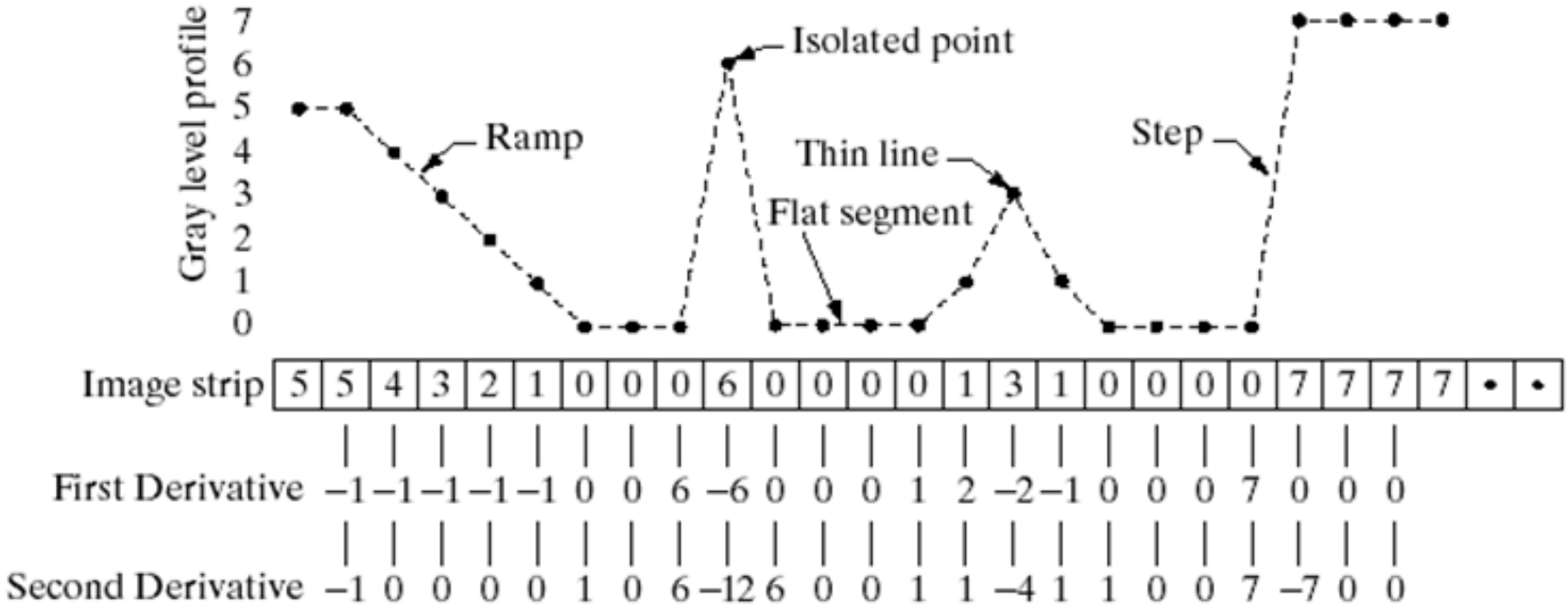
$$\frac{\partial f}{\partial x} = f(x+1) - f(x)$$



1st Order & 2nd Order Derivative

$$\frac{\partial f}{\partial x} = f(x+1) - f(x)$$

$$\frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x)$$



THANK YOU!!!